Spatio-Temporal Trust Inference Closes Two Structural Blind Spots of Window-Based Trust Aggregation in Low-Altitude UAV Networks

Anonymous Authors

Department of Computer Science and Engineering, Anonymous Institution

Correspondence: to be provided after peer-review anonymization is removed.

Abstract: Trust scores are useful in a cooperative UAV mission only when they inform an operational decision, and the decision begins with what the aggregator is able to see. We construct five normalized evidence streams—behavior, communication, energy, task execution, and history—and study what a trust aggregator must represent in order to catch an attacker that it cannot see directly. The finding is a structural blind spot in the aggregation step rather than a deficiency of the classifier, and it takes two independent forms. The first is temporal. The marginal window summaries that conventional trust aggregation computes leave periodic on–off attackers at 0.0–38% detection across five rule-based comparators, while adding five scale-free temporal-shape statistics lifts the same models to 88.0–92%, a +50.0 to +92.0 pp gap that appears only on the one attack class the aggregators are structurally unable to express; the gap survives a duty-cycle sweep, with the rule-based aggregator collapsing from 42.7% to 0.0% detection between a 50% and a 5% duty while the shape-aware methods degrade from 92.7% to 12.0%. The second is spatial, and no amount of temporal detail addresses it. An attacker that reports well-formed telemetry indistinguishable from the benign population is invisible to every method that scores one node at a time: with the marginal and shape blocks alone, detection of such a node is 2.1%, and the learned per-node baselines detect 6.2% and 4.2%. Adding a five-dimensional neighbourhood-relative block, which compares a node against the link conditions its neighbours actually experience, raises detection to 83.6%, a +81.5 pp paired gain ($t = 63.7$, $p < 0.0001$), while the same vector leaves the per-node baselines where they were. The two blocks are disjoint modalities rather than redundant descriptions of one: the marginal block detects 93.2% of constant attackers but only 62.6% of on–off ones, the shape block inverts that pattern, and the neighbourhood block is the only one that sees the behaviourally-clean attacker, whose own features have effect sizes below 0.04 against the benign population while the neighbourhood-relative statistics reach 1.99. We instantiate the finding in S-TIM (Spatio-Temporal Trust Inference Model), a supervised encoder over the marginal, shape, and neighbourhood blocks, which exceeds the strongest rule-based comparator by +9.0 pp accuracy and +15.0 pp F-score on paired seeds ($p < 10^{-6}$). In the scenario that contains the behaviourally-clean attacker, the comparison that the published scenario cannot make becomes possible, and S-TIM leads every aggregate metric: 93.8% accuracy and 89.4% F-score against 86.0% and 72.8% for the strongest learned control and 39.7–44.0% for the rule adapters, with paired differences of +7.7 pp and +16.7 pp against the learned control ($p < 0.0001$). The blocks are not free: the shape block costs 8.7 pp on constant attackers and the full representation still costs 3.6 pp on constant and 2.2 pp on dormant attackers, which is reported alongside the gains. The gain belongs to the representation, and the controls identify it as such: with the encoder, the input width and the early-stopping rule held fixed, no arrangement of the auxiliary heads changes the detection rates, and a fusion-mechanism ablation that holds the feature vector fixed finds the multi-head policy and a single nonlinear head equivalent, while a linear head both extracts less of the neighbourhood signal and pays 6.3 pp on the classes it already handled. The executable artifact is deliberately narrower than a full multi-agent reinforcement-learning system: it uses simulator labels during offline training, freezes the model for evaluation, and does not learn from environment rewards or deployment trajectories. The results establish reproducibility and feasibility within the generated trust-stream setting, and the packet-level evidence is reported in full, including the part that does not favour the proposed model: on five multi-seed ns-3 traces of forty UAVs, in which the attacker's signature is a level shift that a threshold on the level already separates, four of the five rule aggregators reach 0.98–0.99 accuracy against 0.93 for the proposed model, the fifth flagging four nodes in five, and the scenario contains no level-indistinguishable attacker, so it tests neither blind spot. Against the feature-matched learned control on the same traces the proposed model is ahead at the published operating point by +4.5 pp accuracy ($p = 0.037$) and +10.9 pp F-score ($p = 0.055$), at equal ranking quality within the resolution of five traces. Claims about online adaptation, communication cost, and flight deployment likewise require independent experiments.

Keywords: UAV networks; trust management; task allocation; zero trust; reproducible simulation

1 Introduction

UAV swarms used for inspection, sensing, and emergency response rely on nodes that remain inside the mission after admission. A valid identity is therefore not sufficient evidence of trustworthy behavior. A compromised node can selectively forward traffic, falsify a task result, or alternate between benign and harmful behavior while retaining the privileges granted by its earlier history. Dynamic topology and intermittent links make those behaviors difficult to distinguish from ordinary failures [1, 2].

Trust management addresses this problem by updating a node score from recent observations. Existing schemes use fuzzy fusion, anomaly profiles, provenance, or group decisions [7, 8, 10, 23]. Their scores, however, are often evaluated only as detector outputs. In a mission, the practical question is what changes after a node becomes risky: whether it may receive a sensitive task, how much traffic it may relay, and when it should be isolated. This study examines that narrower trust-to-allocation question.

We present S-TIM (Spatio-Temporal Trust Inference Model) as a reproducible proof of concept rather than a deployment-ready zero-trust architecture. The artifact generates five trust-factor streams, fits a supervised encoder, and uses its risk output in a model-based task-allocation simulation. Although the architecture was motivated by parameterized multi-agent decision making, the released training procedure is not reinforcement learning: there is no sampled return, Bellman target, policy-gradient update, or online interaction with an environment. Calling the executable model MARL would therefore overstate what was implemented.

The blindness described above is specific to attacks that live in a node's own behavior, and it is therefore silent about the attacker that matters most when the adversary knows the aggregator scores single nodes over time. Such a node emits well-formed telemetry, accumulates trust at the rate a benign node does, and can hold it indefinitely; every statistic a per-node aggregator computes is by construction the statistic a benign node in the same radio cell would produce. No amount of temporal detail can separate the two, because the information that separates them is not in the node. This second blind spot is spatial rather than temporal, and closing it requires a different kind of statistic: one that compares a node against the link conditions its neighbors actually experience.

The study asks three bounded questions. First, how should generated trust streams be summarised so that constant, dormant, and on–off attackers are all separable on held-out random seeds? Second, on the same evidence, what does an aggregator that scores one node at a time fail to see, and what does a neighborhood-relative statistic add to it? Third, under the allocation rules encoded in the same simulator, does trust-guided selection improve a task-success proxy? The answer to the first question is that the summarisation step, not the classifier, is the binding constraint: the marginal window aggregates that conventional trust aggregation uses leave periodic on–off attackers at 0.0–38% detection across five rule-based comparators, and five scale-free shape statistics lift the same models to 88.0–92%, a +50.0 to +92.0 pp gap that is concentrated entirely on the one attack class the aggregators are structurally unable to express. We instantiate the finding in S-TIM, a supervised encoder over the marginal block plus the shape block, and show that the representation rather than the network is what closes the gap: the same 18 features given to a plain single-head MLP give the same detection rates, and switching the auxiliary supervision on and off moves nothing, because a probe shows the auxiliary target is already 96.7% recoverable from the marginal features. A paired duty-cycle sweep then shows how far this goes—below a 20% active fraction the rule-based aggregator is functionally blind, identifying none of 150 pooled on–off nodes at a 5% duty, while shape-aware methods still recover a seventh of them. The answer to the second question is that the per-node view is structurally blind to an attacker whose behavior is clean, and that the missing information is recoverable: with the marginal and shape blocks alone such a node is detected at 2.1%, while five neighborhood-relative statistics raise the same model to 83.6% on the same seeds, a +81.5 pp paired gain ($t = 63.7$, $p < 0.0001$), and the learned per-node baselines, which the protocol withholds that block from, stay at 6.2% and 4.2%. Holding the feature vector fixed and varying only how the blocks are combined confirms that this too is a representation result: the multi-head policy and a single nonlinear head are equivalent, while a linear head both extracts less of the neighborhood signal and pays 6.3 pp on the classes it already handled. The answer to the third is suggestive rather than causal because the task simulator contains mechanism-specific rules. A short ns-3-derived trace is included to expose, rather than hide, the remaining packet-level gap.

Problem statement. This paper addresses a trust-layer decision problem for fixed identities: how should a risk estimate derived from noisy multi-factor evidence affect a subsequent task-allocation decision? The empirical scope is a generated trust-stream benchmark. It does not cover admission control, Sybil resistance, real-time distributed execution, flight deployment, or the validation of absolute position integrity; the spatial result below is a statement about what a trust aggregator can and cannot resolve from the trust stream itself, not a position-verification scheme.

Figure 1 gives the intended deployment architecture. Only its trust-stream classifier and allocation proxy are exercised in the released artifact. The contributions are consequently limited to the following:

- a documented generator for behavioral, communication, energy, task, and historical trust streams under three temporal attack patterns and one spatially-inconsistent pattern, in which a subset of the malicious population reports telemetry drawn from the benign distribution while standing in an airspace cell whose RF field differs from the one it reports;
- a characterisation of a second, independent blind spot, which no amount of temporal detail can close: an attacker whose behavior is clean is invisible to every method that scores one node at a time. With the marginal and shape blocks alone it is detected at 2.1%, and the learned per-node baselines detect 6.2% and 4.2%. A five-dimensional neighborhood-relative block raises the same model to 83.6% on the same seeds, a +81.5 pp paired gain ($t = 63.7$, $p < 0.0001$), while the behaviour channels of that attacker remain indistinguishable from benign ones, with effect sizes below 0.04 against the benign population against 1.99 for the neighborhood-relative statistics;
- the headline detector results, measured on paired evaluation seeds: S-TIM exceeds the strongest rule-based comparator by +9.0 pp accuracy and +15.0 pp F-score ($p < 10^{-6}$), and the method-class analysis shows why, with on–off detection rising from 0.0–38% across five rule-based aggregators to 88.0–92% across shape-aware methods. In the spatially inconsistent scenario, in which the comparison is not compressed by a perfectly separable training split, S-TIM leads every aggregate metric with 93.8% accuracy, 89.4% F-score and 88.9% TSR, against 86.0%, 72.8% and 82.2% for the strongest learned control (+7.7 pp, +16.7 pp and +6.7 pp, $p < 0.0001$) and 39.7–44.0%, 44.9–45.5% and 74.5–75.9% for the rule adapters;
- a controlled identification of the mechanism, with a reproducible supervised encoder and feature-matched LR/MLP controls on disjoint training and evaluation seeds: with the encoder, the input width and the early-stopping rule held fixed, switching every auxiliary loss off, supervising only the attack-modality head, or supervising all four changes no detection metric, and an information probe shows why, since the auxiliary target is already 96.7% recoverable from the marginal features alone. A fusion-mechanism ablation that hands the identical 23-dimensional vector to the linear head, the single MLP head, and the multi-head policy reaches the same conclusion from the other direction: the two nonlinear learners are equivalent, the linear head extracts less of the neighborhood signal and pays 6.3 pp on constant attackers for what it gains, and the gain is therefore a property of the representation rather than of the network;

The remainder of this paper reviews related work, defines the intended architecture and executable subset, reports the trust-stream experiments and a multi-seed packet-level validation, and closes with limitations.

2 Related Work

Trust evaluation for UAV networks now includes reputation models, evidence fusion, blockchain support, and learning-based designs. Yet low-altitude UAV studies rarely integrate zero-trust verification, distributed trust learning, uncertainty aggregation, and mission incentives. We review four related areas: UAV trust management, position-aware trust and integrity monitoring, zero-trust assessment, and MARL for UAV networking and security.

2.1 Trust Management in UAV and FANET Systems

Recent surveys show that trust management has become a core security primitive in FANETs, where dynamic topology, intermittent links, and resource limits make purely identity-based protection insufficient

[1–5]. Existing schemes fall into three categories: behavior-centric trust analytics, distributed or blockchain-supported trust, and learning-enhanced trust evaluation.

Behavior-centric trust evaluation widely uses fuzzy and evidence-fusion methods to combine heterogeneous observations. Benfriha et al. proposed FUBA, which uses fuzzy logic to integrate indicators such as RSSI, packet delivery ratio, delay, battery level, and humidity to identify insider threats in FANETs [8]. Li et al. developed an adaptive multi-granularity trust management scheme for UAV visual sensor security under adversarial attacks, where coarse- and fine-grained evidence is fused to improve robustness against known and unknown threats [9]. Akram et al. introduced GALTrust, which combines generative adversarial learning with type-2 fuzzy logic to improve adaptability to emerging malicious behaviors in spatial crowdsourcing drone services [10]. More recent federated-security studies also move trust assessment toward collaborative anomaly detection, including FL-based intrusion detection for FANETs, multimodal swarm anomaly detection, and communication-efficient FL for resource-constrained UAV-IoT systems [11–13]. Moving to task offloading, Kumar et al. evaluate trust for UAV-as-a-Service by combining residual energy, achievable data rate, and a reputation score that is transferred between a task and a service UAV through a Rubinstein bargaining game [24]. Their trust value is an energy gate multiplied by a weighted reputation and rate term, so it expresses resource adequacy and prior success rather than adversarial intent, and the paper reports a small improvement in delay, energy, and failure rate with no adversary in the loop. These studies significantly enrich trust evidence modeling. Their remaining limitation for the problem considered here is not that they cannot detect insider behavior at all, but that fuzzy, profile-based, or task-specific fusion can become sensitive to distribution shifts when the malicious population is dense and attack timing is deliberately evasive.

A second line of work improves trust persistence and auditability through distributed storage or hierarchical trust exchange. Ge et al. developed a provenance-aware distributed trust model for resilient UAV networks, highlighting the value of peer-to-peer evidence tracing in hostile environments [7]. Zuo et al. proposed a hierarchical blockchain-based trust measurement method for drone clusters to address trust inheritance, cross-cluster re-evaluation, and historical trust verification under dynamic node entry and exit [6]. Akram et al. proposed TMIoDT, which integrates digital twins, blockchain, and deep-learning-based intrusion detection to secure Open RAN-based Internet of Drone Things applications [17]. Closely related secure-FL frameworks have further explored malicious-node detection and trusted collaboration through blockchain-supported service provisioning, privacy-preserving IoDT security, and hierarchical federated aggregation [18–21]. Xu et al. pair a Bayesian trust assessment for quantum federated learning with Stackelberg incentives for the same UAV-assisted setting [22]. The two parts are separable: the incentive side is a full game-theoretic construction, whereas the trust degree itself is the posterior mean of a Beta–Binomial model, that is, a smoothed cumulative proportion of high-quality model updates. Their reported neutral device, which alternates between good and bad updates, therefore settles at an intermediate trust value rather than converging to a malicious verdict—an outcome the aggregator implies by construction rather than by choice. More recently, Hu et al. proposed GDM-DTM for malicious-node detection in low-altitude UAV networks by combining group decision making with dynamic trust management [23]. These schemes improve distributed credibility, trust persistence, and malicious-node detection. For our setting, the central gap is that the trust state is usually treated as an assessment output rather than as the basis for learned reward, penalty, and task-allocation decisions.

2.2 Position-Aware Trust and Integrity Monitoring in UAV Swarms

A separate line of work attacks the problem from the position side. A UAV that forges its reported location can steer a swarm, corrupt cooperative localization, and carry a payload to a forbidden region while every behavioural statistic it reports remains plausible. This is an active literature. Zhong et al. enhance the resilience of UAV networks against GPS spoofing through a Byzantine distributed detection algorithm [50], and Tariq and Ahamed Ahanger address BeiDou spoofing in decentralized swarms with multi-model fusion [54]; Dang et al. detect spoofing on cellular-connected UAVs with a deep ensemble [51], and Zhao et al. survey the multi-source-fusion detectors that the field has accumulated [53]. On the measurement side, Fang, Han, and Schotten give an information-theoretic account of the performance and security trade-off in trustworthy UAV cooperative localization [52], and Keshavarz et al. build a real-time trust-monitoring framework for a UAV network [55].

Two features of that literature matter for the present question. First, most of these mechanisms need a reference: an absolute position estimate, a trusted reference station, or a second sensor whose own integrity is assumed. The anchor-free alternative is comparatively young. Vijay et al. propose

range-based multi-robot integrity monitoring that is explicitly anchor-free, detecting compromised or faulty robots from the consistency of inter-robot ranges rather than from an absolute reference [49]. Second, these mechanisms operate on the positioning stack, whereas trust management operates on an evidence stream that the positioning stack never sees. The question they leave open is what a trust aggregator can resolve when the only evidence available is the per-node trust stream and no trusted anchor exists at all. Cluster-level trust management in drone swarms [6] inherits the same limitation: it exchanges and hierarchically aggregates trust, but every node it evaluates is judged against its own history and its own report.

The attacker considered here lies precisely in that gap. It does not forge a position and does not need to: it reports well-formed telemetry consistent with the benign population and stands in an airspace cell whose radio conditions differ from the ones it reports, so that only its neighbors disagree with it. No absolute reference is required to detect it, and no absolute reference is available. The question asked here is therefore upstream of the integrity-monitoring literature in the same sense that it is upstream of the on-off-aware trust literature: not which detection rule is better once the right evidence has been identified, but what evidence has to exist in the first place for any rule to work.

2.3 Zero-Trust and Continuous Trust Assessment

Zero trust shifts security from perimeter-based assumptions to continuous, resource-centric verification. The NIST zero-trust architecture defines this paradigm through the principle of "never trust, always verify" and replaces implicit perimeter-based trust with continuous, context-aware verification [15]. Building on this idea, Wang et al. proposed a zero-trust-driven collaborative intrusion detection framework for IoT that uses continuous trust assessment to support distributed security decisions [31]. In the UAV and 6G context, Al Ridhawi and Aloqaily integrated zero trust, digital twins, federated learning, and blockchain into a UAV-enabled 6G framework, highlighting the broader relevance of decentralized trust enforcement in highly dynamic aerial networks [32]. In the UAV authentication domain, Wang et al. introduced a context-aware trust evaluation model for malicious UAV detection during authentication, showing that trust can also be injected into the authentication stage rather than being treated purely as a post hoc reputation signal [14].

Zero trust is particularly relevant to low-altitude UAV swarms: node roles may change during flight, links may appear and disappear rapidly, and local observations may be incomplete or noisy. Trust decisions cannot be based solely on prior membership or static credentials, but must be continuously updated from current observations, device status, communication behavior, and mission context. Therefore, trust decisions should be dynamic, uncertainty-aware, and resource-conscious rather than static or binary.

These studies move trust evaluation closer to continuous verification. Nevertheless, the current zero-trust literature still focuses mainly on authentication, access control, or intrusion detection. It does not fully address how trust should evolve online in low-altitude UAV swarms when heterogeneous evidence from behavior, communication quality, task execution, energy state, and historical interactions must be fused under strict resource constraints. In particular, uncertainty-aware aggregation and decentralized incentive adaptation remain underexplored in zero-trust UAV settings.

2.4 MARL for UAV Networking and Security

MARL has recently become a prominent framework for decentralized decision making in UAV systems. A recent survey by Ekechi et al. reports that MARL has mainly been used for UAV control, formation, coverage, coordination, and cooperative optimization, rather than for trust management itself [16]. Representative examples include the MADDPG-based multi-UAV trajectory planning framework of Wang et al. for UAV-assisted mobile edge computing [33], the semi-distributed multi-agent federated reinforcement learning approach of Nie et al. for resource management in UAV-aided MEC systems [34], and the joint optimization of trajectory control, task offloading, and resource allocation in air–ground integrated networks by Alam and Moh [35]. Closely related learning-and-optimization studies in UAV-enabled federated settings have examined asynchronous scheduling, contract-based participant selection, hierarchical personalized learning, air–ground FL coordination, and energy-aware trajectory and resource management [36–41]. Together, these cited works provide evidence that distributed learning can support optimization under partial observability and dynamic coupling among UAVs. More recently, Cao et al. explored LLM-based multi-agent situation awareness for zero-trust space-air-ground integrated networks

[42]. Their results further suggest that model selection in security-oriented multi-agent systems should balance capability and deployment cost; for specialized subtasks, smaller task-adapted models can be more practical than larger general-purpose models.

However, existing MARL applications in UAV networking remain largely performance driven, with objectives such as delay reduction, energy efficiency, coverage, or offloading efficiency. The reward design in these studies is rarely aligned with trustworthiness, collusion resistance, or long-horizon security incentives. MARL has been used to optimize how UAVs move, communicate, or allocate resources, but not how they collaboratively evaluate trust, calibrate trust-factor importance, and propagate trustworthy decisions under zero-trust assumptions.

Overall, prior work has made strong progress in trust modeling, distributed auditability, and UAV-oriented MARL, but four problem-specific challenges remain insufficiently addressed for low-altitude UAV trust evaluation:

(1) **Accurate trust assessment under high malicious-node ratios**: Fuzzy, profile-based, and group-decision mechanisms can lose discrimination when the malicious population substantially shifts local trust distributions, leading to poor mission continuity.

(2) **Temporal and multi-factor evasion**: Dormant and on-off attackers can exploit historical trust accumulation, while multi-factor manipulation requires deciding which behavioral, communication, energy, task, and history evidence remains reliable at each update. This problem is not open. On-off-aware trust management is established, from the trust model that defends on-off attacks directly [26] through its distributed variant [27] and the smart-middleware detector [28], to the demonstration that cumulative weighted and exponentially weighted moving averages do not work well under on-off attacks and the asymmetric weighting that repairs them [29], and to the recent trust model that criticises sliding-window aggregation for being unable to select an appropriate window length [30]. What these mechanisms share is that they intervene on the aggregated trust value or on the aggregation that produces it: they re-weight it, re-window it, mitigate the damage it does, or index its fragmentation [25], and an index applied after the fact inherits whatever the underlying aggregator has already smoothed away. The question asked here is upstream of all of them, and it is the one they leave open: whether the information an intermittent attacker destroys can be recovered at all from the summaries that a fixed-window aggregator is defined to produce. Our answer is that it cannot be recovered from those summaries, and that the recovery requires different statistics over each channel rather than a different weighting, a different window, or an index applied afterwards.

(3) **Closing the trust-to-action loop**: A detector that outputs only a trust or risk score does not by itself decide how task allocation, incentives, or penalties should change; this is the specific gap exposed by the comparison with the strong risk-only MLP-Trust baseline.

(4) **Spatially inconsistent attackers**: An attacker whose telemetry is drawn from the benign population is invisible to every aggregator that scores one node at a time, because the statistics such an aggregator computes are by construction the statistics a benign node in the same radio cell produces. The missing information is not in the node, so no amount of temporal detail recovers it, and the position-integrity literature that does address this class of attacker relies on an absolute reference that a swarm in infrastructure-scarce conditions does not have. What is needed is a statistic that compares a node against the conditions its neighbors actually experience, which is a question about what a trust aggregator must represent rather than about which detection rule to apply.

The present artifact addresses only part of this gap: it evaluates temporal trust features and a score-based allocation proxy. Multi-agent execution and confidence-aware aggregation remain proposed extensions. In particular, the hybrid action space used here—a continuous factor-weight vector together with a quantized incentive level—is the setting that parameterized deep Q-networks are designed for [47, 48], and a genuine multi-agent learner would additionally optimize the shared objective through bootstrapped value targets under partial observability [43, 46]. The released artifact performs neither: its auxiliary heads are trained as supervised regularizers on targets derived from the class label, and only the risk head is read at evaluation time.

3 System Model

We consider a low-altitude UAV swarm executing a cooperative mission in an infrastructure-scarce environment. This section separates the intended system from the subset implemented in the artifact.

3.1 Problem Definition

At each update interval, the system receives partial and noisy observations of each UAV's behavior, communication quality, energy state, task execution, and historical reputation. The objective is to infer a trust state that reflects the UAV's current behavior and then use that state to regulate future participation through rewards, penalties, and task-allocation preferences. Formally, for each UAV U_k, S-TIM must output a global trust score $\mathcal{T}_k(t)$, a trust class $C_k(t)$, and an incentive action $\pi_{jk}(t)$ for the local Thinker Agent j that supervises U_k:

$$(\mathcal{T}_k(t), C_k(t), \pi_{jk}(t)) = \Pi_\theta(o_j(t), \mathcal{H}_k(t)), \tag{1}$$

where $o_j(t)$ is the partial observation available to agent j, $\mathcal{H}_k(t)$ is the recent trust-evidence history of node k, and Π_θ is the learned trust–incentive policy. The conceptual objective is to improve both malicious-node detection and mission continuity while respecting update-cycle resource budgets:

$$\max_{\Pi_\theta} \mathbb{E}[\lambda_1 F_1 + \lambda_2 \mathrm{TSR} - \lambda_3 C_{\mathrm{comm}} - \lambda_4 T_{\mathrm{eval}}], \tag{2}$$

subject to bounded trust scores and an incentive budget. This is a conceptual objective only. The empirical model minimizes the supervised loss in Section 3.5.3; it does not optimize F_1, TSR, communication cost, or latency jointly.

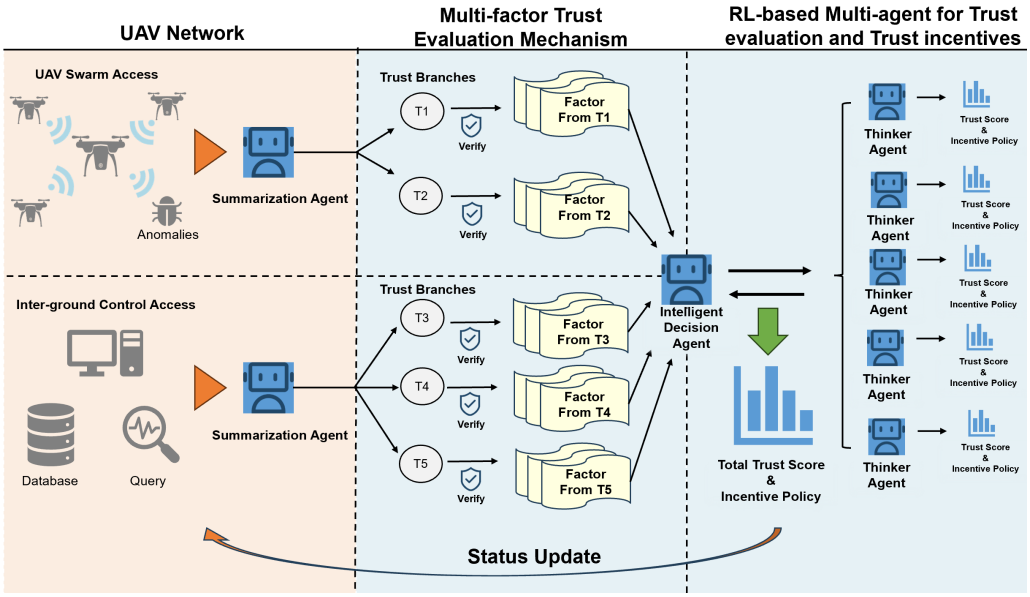


Figure 1: Intended S-TIM deployment architecture. The released artifact implements the T1–T5 trust-stream features, a shared supervised network, and a model-based allocation proxy. Parallel on-board agents, confidence-weighted fusion, and a closed deployment feedback loop remain design elements rather than experimentally validated components.

3.2 UAV Network Model

We model the low-altitude UAV network as a dynamic ad hoc network of N heterogeneous UAV nodes, $\mathcal{U} = \{U_1, U_2, \ldots, U_N\}$. Each node U_i is characterized by a tuple $\langle \mathbf{p}_i(t), v_i(t), E_i(t), C_i \rangle$, where $\mathbf{p}_i(t) \in \mathbb{R}^2$ is the 2D position at time t, $v_i(t) \in [v_{\min}, v_{\max}]$ is the velocity, $E_i(t) \in [0, E_i^{\max}]$ is the residual energy, and C_i denotes the computational capacity (CPU cycles/s). Nodes may enter or leave the swarm during the mission according to operational requirements.

The communication topology is a time-varying undirected graph $G(t) = (\mathcal{U}, \mathcal{E}(t))$, where an edge $(i, j) \in \mathcal{E}(t)$ exists iff the Euclidean distance satisfies $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2 \le R_{\mathrm{comm}}$, with $R_{\mathrm{comm}} =$ 300 m (Table 2). Each link (i, j) has a time-varying quality characterized by packet delivery ratio $\mathrm{PDR}_{ij}(t)$, received signal strength $\mathrm{RSSI}_{ij}(t)$, and one-way latency $L_{ij}(t)$. The network is *infrastructure-scarce*: no fixed base stations or ground controllers are assumed to be continuously reachable; all trust computation and decision-making must execute on-board or via peer-to-peer exchange.

3.3 Threat Model

We use a fixed-identity synthetic adversary model. It is not adaptive to the learned parameters.

Adversary capabilities. A subset of nodes $\mathcal{M} \subset \mathcal{U}$ ($|\mathcal{M}|/N \in [0.05, 0.50]$ in our experiments) is controlled by an adversary. Each malicious node $U_m \in \mathcal{M}$ can independently manipulate any subset of the five trust dimensions by:

- **T1 (Behavioral)**: deviating from planned flight paths, skipping scheduled heartbeats, or responding inconsistently to coordination messages;
- **T2 (Communication)**: performing selective forwarding (dropping a fraction $p_{\text{drop}} \in [0.3, 1.0]$ of packets), injecting spurious traffic, or manipulating transmission power to create asymmetric links;
- **T3 (Energy)**: exhibiting anomalous consumption patterns (e.g., sudden depletion through jamming, or unusually low consumption by remaining idle during active mission phases);
- **T4 (Task)**: returning fabricated or degraded task results (e.g., falsified sensor readings, partial payload delivery);
- **T5 (Historical)**: this dimension is not directly manipulable by the adversary—it is an accumulated reputation signal maintained by S-TIM—but its update is indirectly affected when the adversary manipulates T1–T4.

Attack strategies. We consider four canonical attack patterns, three temporal and one spatial, that cover the spectrum from naive to evasive adversaries:

(i) **Constant attack**: the malicious node exhibits adversarial behavior from $t = 0$ and maintains it throughout the mission. This models unsophisticated attackers and serves as a lower bound on detection difficulty.

(ii) **Dormant attack**: the malicious node behaves honestly during an initial warm-up period ($t \leq T_{\text{warm}}$), accumulating trust, then switches to adversarial behavior. This models adversaries that attempt to evade initial screening.

(iii) **On-off attack**: the malicious node toggles between honest and adversarial behavior with period T_{toggle}, attempting to keep its trust score near the classification boundary where decisions are most uncertain.

(iv) **Spatially inconsistent attack**: the malicious node reports channels drawn from the benign population, so that every behavioural statistic computed on the node is statistically indistinguishable from a benign node's, and it is placed in an airspace cell whose radio field differs from the one it reports. Its neighbours carry the local field offset that it does not. The attack is invisible to any aggregator that scores one node, and visible only to a statistic that compares a node against the conditions its neighbourhood actually experiences.

In mixed-attack experiments, the temporal patterns are assigned cyclically across malicious nodes, and the spatially inconsistent pattern is applied to a configurable fraction of them. The artifact does not implement colluding peer reports, Sybil identities, an adaptive attacker, or training-data poisoning.

Adversary knowledge. The generated attacker knows neither the fitted parameters nor the decision threshold. Because it does not respond to model outputs, the experiments measure generalization across random seeds from one generator, not robustness to a strategic white-box adversary.

Spatial field. The spatial scenario superimposes a smooth channel-quality field over the airspace, interpolated from a coarse random grid at the node positions. Two nodes within communication range therefore observe correlated conditions, which is what gives a neighbourhood any information to offer: the absolute level of a channel now says something about where a node is as well as about what it is doing. The field is applied as a common mode over all four evidence channels rather than to a subset of them, so that the cross-channel gap features remain field-invariant and continue to measure what they are meant to measure. The benign reference is shifted downward by a configurable amount so that the field's excursions clip symmetrically against the lower and upper bounds; without that shift the clipped

benign mean drifts away from the reference, and a node that carries no field offset would become visible to a purely behavioural method for a reason that has nothing to do with its behavior. A fraction of the malicious population is then placed in a cell whose field value differs from the one it reports, and the field offset that node would carry is removed from its own channels while its neighbours keep theirs. Both corrections are diagnostics rather than tuning: each was found by a script in the released analysis directory, and each was verified to leave the published configuration untouched.

Design implications. The generator mainly tests whether temporal summaries of T1 and T4 separate the three temporal attack profiles from benign nodes, and whether a neighbourhood-relative statistic separates the fourth. It does not test majority collusion or Byzantine aggregation, and the spatially inconsistent attacker is generated by the same machinery rather than by an adversary that adapts to the model. Any deployment would still require cryptographic identity binding and an independently validated observation pipeline.

3.4 Multi-factor Trust Evaluation Mechanism

S-TIM represents credibility with five trust dimensions. They are not statistically independent, and the present study does not support an "exponentially harder" evasion claim. T5 is explicitly computed from the other four dimensions and is therefore a memory term, not a separate sensor.

3.4.1 Trust Factor Decomposition

T1 denotes behavioral consistency, T2 communication reliability, T3 energy behavior, and T4 task quality. T5 is an exponentially smoothed mean of T1–T4. Equations 3–6 describe how these quantities could be obtained from telemetry. In the main experiment, however, the artifact samples normalized T1–T4 values directly from attack-dependent distributions and computes T5 from those samples; it does not simulate the raw packet, position, energy, or task variables in these equations.

As illustrated in Figure 1, the trust evaluation is decomposed into multiple trust branches, each corresponding to a distinct trust factor. Let the overall trust score of UAV node i at time t be denoted as $T_i(t) \in [0, 10]$, which is a function of five fundamental trust dimensions (the formal aggregation mechanism is presented in Section 3.4.2), where each trust branch $T_i^{(m)}(t) \in [0, 1]$ is defined as follows:

- **T1: Behavioral Trust** $T_i^{(1)}(t)$. This factor evaluates operational consistency, including adherence to flight paths, communication regularity, and response reliability:

$$T_i^{(1)}(t) = \frac{1}{N_b} \sum_{j=1}^{N_b} \mathbb{I}\Big(a_{i,j}(t) = a_{i,j}^{expected}(t)\Big) \tag{3}$$

  where $\mathbb{I}(\cdot)$ is the indicator function, $a_{i,j}(t)$ represents the actual action in behavior category j, and $a_{i,j}^{expected}(t)$ is the expected behavior. This yields $T_i^{(1)}(t) \in [0, 1]$.

- **T2: Communication Trust** $T_i^{(2)}(t)$. This factor assesses link quality and stability through packet delivery ratio, signal strength, and latency:

$$T_i^{(2)}(t) = w_p \cdot PDR_i(t) + w_s \cdot \frac{RSSI_i(t)}{RSSI_{max}} + w_l \cdot \left(1 - \frac{L_i(t)}{L_{max}}\right) \tag{4}$$

  where $PDR_i(t)$, $RSSI_i(t)/RSSI_{max}$, and $1 - L_i(t)/L_{max}$ are each normalized to $[0, 1]$, and $w_p = 0.4, w_s = 0.3, w_l = 0.3$ are weighting coefficients with $w_p + w_s + w_l = 1$. Thus $T_i^{(2)}(t) \in [0, 1]$.

- **T3: Energy Trust** $T_i^{(3)}(t)$. This factor monitors energy-consumption patterns and remaining battery life to detect anomalies indicative of malfunction or compromise:

$$T_i^{(3)}(t) = \frac{E_{remaining}(t)}{E_{initial}} \cdot \exp\left(-\alpha \cdot \left|C_i(t) - \bar{C}(t)\right|\right) \tag{5}$$

  where $\frac{E_{remaining}(t)}{E_{initial}} \in [0, 1]$ and the exponential term with $\alpha = 0.5$ is also bounded in $[0, 1]$, yielding $T_i^{(3)}(t) \in [0, 1]$.

- **T4: Task Execution Trust** $T_i^{(4)}(t)$. This factor measures the success rate and quality of task completion, such as sensing, data relaying, or payload delivery:

$$T_i^{(4)}(t) = \frac{1}{N_t} \sum_{k=1}^{N_t} \left(\frac{Q_{i,k}(t)}{Q_k^{max}} \cdot \mathbb{I}(\text{task_success}) \right) \tag{6}$$

  where each term is normalized to $[0, 1]$, so $T_i^{(4)}(t) \in [0, 1]$.

- **T5: Historical Trust** $T_i^{(5)}(t)$. This factor incorporates long-term reputation scores and past behavior records to preserve temporal continuity in trust evaluation:

$$T_i^{(5)}(t) = \beta \cdot T_i^{(5)}(t-1) + (1-\beta) \cdot T_i^{inst}(t) \tag{7}$$

  where both the historical and instantaneous components are in $[0, 1]$, and the smoothing factor $\beta = 0.7$. Thus $T_i^{(5)}(t) \in [0, 1]$.

3.4.2 Multi-factor Trust Aggregation

Each trust branch independently computes a sub-score $T_i^{(m)}(t) \in [0, 1]$ based on real-time data streams and historical logs. These sub-scores are then aggregated into a global trust score $T_i(t) \in [0, 10]$ through a weighted fusion mechanism:

$$T_i(t) = 10 \cdot \sum_{m=1}^{5} \omega_m(t) \cdot T_i^{(m)}(t) \tag{8}$$

where the weights sum to one. The network learns an auxiliary weight head, but the released classifier computes risk directly from 13 summary features and does not use Eq. 8 to obtain its prediction.

3.4.3 Cross-Validation Mechanism

The multi-factor design ensures that no single point of failure or manipulation can compromise the overall trust decision. To enhance robustness, a **cross-validation mechanism** is implemented between trust branches. For any pair of trust factors m and n, the consistency metric is defined as:

$$\Gamma_{m,n}(t) = \left| T_i^{(m)}(t) - T_i^{(n)}(t) \right| \tag{9}$$

If $\Gamma_{m,n}(t) > \gamma_{\text{th}}$, where the cross-validation threshold $\gamma_{\text{th}} = 0.3$ (determined via sensitivity analysis over $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ with 0.3 yielding the best F-score on a validation subset), a potential inconsistency is flagged, triggering deeper inspection. For instance, even if a UAV exhibits normal communication behavior ($T_i^{(2)}(t)$ high), but shows abnormal energy depletion ($T_i^{(3)}(t)$ low), the system can still flag it as potentially compromised. This layered verification keeps trust continuously evaluated rather than assumed from any single evidence channel.

Furthermore, the framework supports **anomaly detection** through statistical analysis of trust factor deviations:

$$\mathcal{A}_i(t) = \mathbb{I}\left(\sum_{m=1}^{5} \left(\frac{T_i^{(m)}(t) - \mu_m(t)}{\sigma_m(t)} \right)^2 > \chi^2_{\alpha,5} \right) \tag{10}$$

where $\mu_m(t)$ and $\sigma_m(t)$ are the mean and standard deviation of trust factor m across the swarm, $\chi^2_{\alpha,5}$ is the chi-squared threshold with 5 degrees of freedom at significance level $\alpha = 0.05$. This redundancy and diversity in trust evidence form the foundation for robust and adaptive trust management in UAV swarms, enabling the detection of sophisticated attacks that may manipulate multiple dimensions simultaneously.

3.5 Spatio-Temporal Risk Model and Allocation Proxy

The intended architecture divides observation across local agents and forwards their estimates to a decision node. The executable artifact is simpler. It applies one shared supervised network independently to each node's diagnostic feature vector. Agent assignments do not enter the feature extractor, and neither confidence-weighted fusion nor budget projection is executed. We retain the broader formulation below to define a possible deployment target, but distinguish it from the evaluated model.

3.5.1 Dec-POMDP Elements

We consider a UAV swarm consisting of N UAV nodes. The trust management system employs M Thinker Agents, where each agent $j \in \{1, 2, \ldots, M\}$ is responsible for a subset of UAV nodes. The overall system can be modeled as a Dec-POMDP (decentralized partially observable Markov decision process) [44]:

$$\mathcal{G} = \langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle \tag{11}$$

where $\mathcal{S}$ is the global state space, $\mathcal{O}$ is the joint observation space, $\mathcal{A}$ is the joint action space, $\mathcal{P}$ is the state transition probability, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

State Space: The state $s \in \mathcal{S}$ at time t is defined as the tuple of all UAV nodes' multi-factor trust scores:

$$s(t) = (\mathbf{T}_1(t), \mathbf{T}_2(t), \ldots, \mathbf{T}_N(t)),$$

where $\mathbf{T}_k(t) = [T_k^{(1)}(t), T_k^{(2)}(t), T_k^{(3)}(t), T_k^{(4)}(t), T_k^{(5)}(t)]$ denotes the five-dimensional trust vector for node k, with each component in $[0, 1]$. The corresponding global trust score $\mathcal{T}_k(t) \in [0, 10]$ is computed by the Intelligent Decision Agent as described in Section 3.5.3.

Observation Space: Each Thinker Agent j observes a partial view of the global state, consisting only of the local trust sub-scores for the nodes in its supervised subset $\mathcal{N}_j$: $o_j(t) = (\mathbf{T}_k(t))_{k \in \mathcal{N}_j} \cup (c_{jk}(t))_{k \in \mathcal{N}_j}$, where $c_{jk}(t)$ is the confidence score assigned by agent j to its own assessment of node k.

Action Space: The action $a_j(t) \in \mathcal{A}_j$ for Thinker Agent j is a hybrid parameterized action consisting of (1) a continuous trust-weight vector $\boldsymbol{\omega}_j(t) = [\omega_j^{(1)}(t), \ldots, \omega_j^{(5)}(t)] \in \Delta^4$ on the probability simplex, and (2) a discrete incentive allocation vector $\boldsymbol{\pi}_j(t) = [\pi_{jk}(t)]_{k \in \mathcal{N}_j}$ specifying the incentive level for each supervised node. Incentive levels are quantized into five bins $\{-2, -1, 0, 1, 2\}$, corresponding to strong penalty, weak penalty, neutral allocation, weak reward, and strong reward. Thus, trust evaluation provides the state used to judge behavior, while the learned policy decides whether the node should be rewarded with more favorable task participation or penalized through restricted access. If a sampled incentive vector violates the cycle budget $B_{\text{total}}(t)$, incentives are projected greedily by retaining actions with the largest expected utility gain per unit cost until the budget is satisfied.

Evaluated architecture: Training is centralized and supervised. Simulator labels determine the risk, action, and value targets. At inference, labels are absent, but the artifact does not emulate decentralized message exchange; it evaluates all per-node feature vectors in one process.

State Transition: The state transition $\mathcal{P}$ is defined implicitly through the trust update equations (Eqs. 25–26) in Section 3.5.4, which govern how trust scores evolve based on observations, incentives, and temporal decay.

3.5.2 Executable Encoder and Its Auxiliary Heads

The implemented encoder consumes per-node diagnostic features extracted from the post-warm-up trust streams. The first block contains thirteen *marginal* statistics: recent weakness in T1, T2, T4, and T5; low-quantile T1/T4 evidence; low-score duration; T1/T4 amplitude and standard deviation; positive degradation slope; and directional cross-factor gaps. Every one of them is an aggregate over a fixed trailing window, so two nodes with identical window summaries receive identical feature vectors regardless of how differently their histories are shaped. A periodic attacker is summarised exactly like a noisy one.

The second block adds five *shape* statistics that the marginal set cannot express, computed per node on channels T1 and T4 and averaged over the two: the mean and maximum absolute excess of the autocorrelation function over its own lag-1 value across lags 3–120; the sign-change rate of the first difference; the ratio $(q_{75} - q_{25})/(q_{95} - q_{05})$, which rises only when a series actually occupies two plateaus; and the coefficient of variation of the intervals between sign changes, which separates a scheduled attacker from a randomly triggered one. No attack period is assumed anywhere in these definitions: no lag of 45 or 90 appears, and every statistic is scale-free, so the network is free to select whatever resolution the data supports rather than an assumed constant.

A third block, used in the spatial scenario of Section 4.6, adds five *neighborhood-relative* statistics. Let A be the row-normalised neighbour matrix of the communication graph, so that $(Ax)_k$ is the mean of x over the nodes within communication range of k, and an isolated node has an all-zero row. The block contains the deviation of the node's own recent T1 and T4 levels from their neighbourhood means; the deviation of its cross-channel gap $(T1 - T4)$ from the neighbourhood mean of that gap; the difference

between the magnitude of its own T1 deviation and the mean magnitude of its neighbours' deviations, which asks whether the node stands out relative to how much its neighbours stand out; and the difference between its own T1 dispersion and the neighbourhood mean dispersion. All five are differences rather than levels, so a node whose absolute behaviour is unremarkable but locally inconsistent still registers, and a node with no neighbours is assigned zeros, because it has no neighbourhood and no spatial evidence can be computed for it. No absolute position is used and no trusted reference is assumed: the block compares a node against the conditions its neighbours actually experience, which is what allows it to see an attacker whose own channels are indistinguishable from benign ones.

The same feature mapping is used by S-TIM and by the learned baselines introduced in Section 4.1, so all comparisons in Table 3 hold the input information fixed. The neighborhood block is the one exception, and the exception is deliberate: LR-Trust and MLP-Trust are per-node aggregators by construction, and handing them a neighborhood-relative feature would turn the comparison into a statement about input width rather than about the aggregator. They are therefore evaluated without it in the main protocol, and a dedicated ablation in Section 4.7 hands them the identical 23-dimensional vector so that the difference between the learners can be attributed to how the blocks are combined rather than to what was available. The 13-feature variant is preserved as `configs/legacy_13dim_features.json` for exact reproduction of the earlier table.

For node k, let $\mathbf{x}_k \in \mathbb{R}^d$ denote the standardized diagnostic feature vector, with $d = 18$ in the published configuration and $d = 23$ in the spatial scenario. A shared encoder maps $\mathbf{x}_k$ to a hidden representation h_k:

$$h_k = f_\theta(\mathbf{x}_k). \tag{12}$$

Four output heads are then computed:

$$\begin{aligned} p_k &= \sigma(g_r(h_k)), \\ \omega_k^{(m)} &= \frac{\exp(g_\omega^{(m)}(h_k))}{\sum_{\ell=1}^{5} \exp(g_\omega^{(\ell)}(h_k))}, \\ \mathbf{q}_k^\pi &= g_\pi(h_k), \\ v_k &= g_v(h_k), \end{aligned} \tag{13}$$

where p_k is the malicious-risk probability, $\boldsymbol{\omega}_k$ is an auxiliary trust-factor weight vector, $\mathbf{q}_k^\pi$ contains five supervised action scores, and v_k is an auxiliary service-value estimate. Only p_k is consumed by the released evaluation path; the auxiliary heads are trained and logged but do not determine the reported task-allocation probabilities. The local diagnostic trust score is

$$\hat{T}_{jk} = 10(1 - p_k), \tag{14}$$

and a node is flagged as malicious when $p_k > 0.60$. A potential deployment could select an incentive bin as

$$\pi_{jk}^{raw} = \arg \max_{\ell \in \{-2,-1,0,1,2\}} q_{k,\ell}^\pi, \tag{15}$$

followed by a budget projection. That projection is not implemented in the current artifact.

3.5.3 Supervised Training Objective

Let $y_k \in \{0, 1\}$ be the simulator label used during training. Let $\boldsymbol{\omega}_k^*$ be a heuristic weight target derived from the diagnostic features, π_k^* an action target derived from y_k and a risk hint, and v_k^* a service-value target derived directly from y_k. The network minimizes

$$\begin{aligned} \mathcal{L}(\theta) = \mathrm{BCE}_w(y_k, p_k) &+ 0.25 \, \|\boldsymbol{\omega}_k - \boldsymbol{\omega}_k^*\|_2^2 \\ &+ 0.20 \, \mathrm{CE}(\pi_k^*, \mathbf{q}_k^\pi) + 0.10 \, (v_k - v_k^*)^2 . \end{aligned} \tag{16}$$

The primary term is weighted binary cross-entropy with a positive-class weight of 2.0. There is no reward return or temporal-difference target in Eq. 16; the value target is 0.8 for benign nodes and -0.4 for malicious nodes. The action target also depends on the ground-truth class. Training uses seeds 0–9 and a fixed decision threshold of 0.60. Evaluation seeds 1000–1049 are not used for fitting. The method is therefore a multi-task supervised classifier, not an offline or online reinforcement-learning algorithm.

The following aggregation equations specify an unevaluated extension for deployments in which several agents observe the same node. They are not invoked by the released Python evaluation.

Total Trust Score Aggregation: The Intelligent Decision Agent computes the total trust score $\mathcal{T}_k(t)$ for each UAV node k by combining the individual trust scores from all Thinker Agents that have observed or interacted with node k:

$$\mathcal{T}_k(t) = \frac{\sum_{j=1}^{M} \phi_{jk}(t) \cdot \hat{T}_{jk}(t)}{\sum_{j=1}^{M} \phi_{jk}(t)} \tag{17}$$

where $\phi_{jk}(t) \in [0,1]$ is the confidence weight assigned to Thinker Agent j's assessment of node k, computed based on the agent's historical accuracy:

$$\phi_{jk}(t) = \frac{\exp\left(-\gamma \cdot \varepsilon_{jk}(t)\right)}{\sum_{j'=1}^{M} \exp\left(-\gamma \cdot \varepsilon_{j'k}(t)\right)} \tag{18}$$

with $\varepsilon_{jk}(t) = \frac{1}{t}\sum_{\tau=1}^{t} \left|\hat{T}_{jk}(\tau) - T_{jk}^{obs}(\tau)\right|$ representing the mean absolute error of agent j for node k over time, and $\gamma = 2.0$ a scaling parameter.

Dispersion measure: A deployment may quantify disagreement among local estimates as

$$\sigma_k^2(t) = \frac{\sum_{j=1}^{M} \phi_{jk}(t) \left(\hat{T}_{jk}(t) - \mathcal{T}_k(t)\right)^2}{\sum_{j=1}^{M} \phi_{jk}(t)} + \sigma_0^2 \tag{19}$$

where $\sigma_0^2 = 0.1$ is a variance floor. This weighted dispersion is not a Bayesian posterior variance unless a probabilistic observation model is supplied.

Final Trust Classification: Based on the total trust score and uncertainty, the Intelligent Decision Agent classifies nodes into trust categories:

$$C_k(t) = \begin{cases} \text{Trusted} & \text{if } \mathcal{T}_k(t) \geq \theta_{\text{high}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \\ \text{Suspicious} & \text{if } \theta_{\text{low}} \leq \mathcal{T}_k(t) < \theta_{\text{high}} \text{ or } \sigma_k(t) > \sigma_{\text{th}} \\ \text{Malicious} & \text{if } \mathcal{T}_k(t) < \theta_{\text{low}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \end{cases} \tag{20}$$

where θ_{high} and θ_{low} are trust thresholds, and σ_{th} is the uncertainty threshold.

A future decision node could map global trust assessments to resource-allocation actions:

$$\mathbf{\Pi}(t) = \{\pi_{jk}(t) : j \in [1, M], k \in \mathcal{N}_j\}, \tag{21}$$

where $\pi_{jk}(t) \in \mathcal{A}_{\text{inc}} = \{-2, -1, 0, 1, 2\}$ is the incentive action for node k by agent j, encoding strong penalty (-2) through strong reward $(+2)$. The policy is determined by solving a constrained optimization that maximizes expected network-wide utility:

$$\max_{\mathbf{\Pi}(t)} \mathbb{E}\left[\sum_{k=1}^{N} U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t))\right], \tag{22}$$

subject to the budget constraint $\sum_{j=1}^{M}\sum_{k\in\mathcal{N}_j} c(\pi_{jk}(t)) \leq B_{\text{total}}(t)$, where $c(\pi) = |\pi|$ quantifies the resource cost of each incentive action. The per-node utility function is:

$$U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t)) = \mathcal{T}_k(t) \cdot \big(1 + \delta \cdot \mathbb{I}(\pi_{\cdot k}(t) > 0)\big) - \kappa \cdot \mathbb{I}\big(\mathcal{T}_k(t) < \theta_{\text{low}}\big), \tag{23}$$

with $\delta = 0.2$ and $\kappa = 2.0$. These equations are design specifications. The current task-success simulator instead converts each method's score to a sampling weight and applies a stronger down-weighting rule to S-TIM; it does not solve Eq. 22.

3.5.4 State Update Mechanism for Trust Evolution

The following state update is a proposed deployment mechanism, not the recurrence used in the main generator. In the artifact, T5 follows $T_5(t) = 0.84T_5(t-1) + 0.16\bar{T}_{1:4}(t)$ and the trained risk model is

evaluated on summary windows. Incentive outputs do not alter later trust streams. A possible deployment transition is

$$\mathbf{T}(t+1) = f\left(\mathbf{T}(t), \mathbf{A}^{inc}(t), \mathbf{A}^{task}(t), \mathbf{E}(t)\right) \tag{24}$$

where $\mathbf{T}(t) \in \mathbb{R}^N$ is the vector of global trust scores, $\mathbf{A}^{inc}(t)$ is the incentive action matrix, $\mathbf{A}^{task}(t)$ captures task execution outcomes, and $\mathbf{E}(t)$ represents environmental factors. The trust score for each node k evolves according to the following three-stage process:

Temporal Trust Decay: To prevent trust stagnation and enforce continuous verification, we incorporate a temporal decay factor:

$$\tilde{\mathcal{T}}_k(t+1) = \mathcal{T}_k(t) \cdot \exp\left(-\rho \cdot \Delta t\right) \tag{25}$$

where $\rho = 0.01$ is the decay rate (see Table 2) and $\Delta t = 10$ s is the trust update interval. This ensures that trust gradually diminishes without positive reinforcement.

Trust Update Based on Observations: The final trust score is updated via a Bayesian filter that combines the decayed prior with new observations:

$$\begin{aligned} \mathcal{T}_k(t+1) &= a_k(t)\,\tilde{\mathcal{T}}_k(t+1) + \big(1 - a_k(t)\big) \\ &\quad \cdot T_k^{obs}(t+1), \\ a_k(t) &= \frac{\sigma_{obs}^2}{\sigma_k^2(t) + \sigma_{obs}^2}. \end{aligned} \tag{26}$$

where $\sigma_{obs}^2 = 0.5$ is the observation noise variance, and $T_k^{obs}(t+1)$ is the aggregated observed trust score from all Thinker Agents. This form assigns more weight to new observations when the prior uncertainty $\sigma_k^2(t)$ is high, and more weight to the decayed prior when the current estimate is confident. The observation term is:

$$T_k^{obs}(t+1) = \frac{1}{|\mathcal{M}_k(t+1)|} \sum_{j \in \mathcal{M}_k(t+1)} T_{jk}^{obs}(t+1) \tag{27}$$

with $\mathcal{M}_k(t+1)$ denoting the set of agents that observed node k at time $t+1$.

Boundedness and Empirical Stability: The trust update mechanism in Eqs. 25–26 admits bounded updates under the $[0, 10]$ trust scale. Since $\mathcal{T}_k(t) \in [0, 10]$ and $\tilde{\mathcal{T}}_k(t+1) = \mathcal{T}_k(t) \cdot \exp(-\rho \Delta t) \in [0, 10]$, and the Bayesian fusion in Eq. 26 is a convex combination of two bounded terms (with coefficients in $[0, 1]$ that sum to one), the trust score remains in $[0, 10]$ for all t, preventing numerical divergence.

We do not claim convergence of the Dec-POMDP or of the generated trajectories. The boundedness argument applies only to the proposed convex update.

Scope of the update claim: Equations 25–26 show boundedness of the proposed update only. They do not establish convergence, and the released simulator does not close the action-to-future-state loop.

3.5.5 Evaluation Metrics

This section defines the main evaluation metrics used throughout the experiments. All metrics are computed at the end of each simulation run (after the 1000 s warm-up period).

Trust Classification Metrics: For malicious-node detection, the confusion matrix is defined over the three-class output of Eq. 20, with Suspicious mapped to Malicious for binary evaluation. Let TP, TN, FP, FN denote true/false positives/negatives, respectively. Detection accuracy, precision, recall, and F_1-score are:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, & \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, & F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \tag{28}$$

Ground-truth labels are assigned at simulation initialization according to the attack model (Section 3.3) and are never observable by the Thinker Agents during execution.

Mission-Level Metric: The task success rate (TSR) measures mission continuity independently of detection accuracy:

$$\text{TSR} = N_{\text{succ}} \,/\, N_{\text{all}}, \tag{29}$$

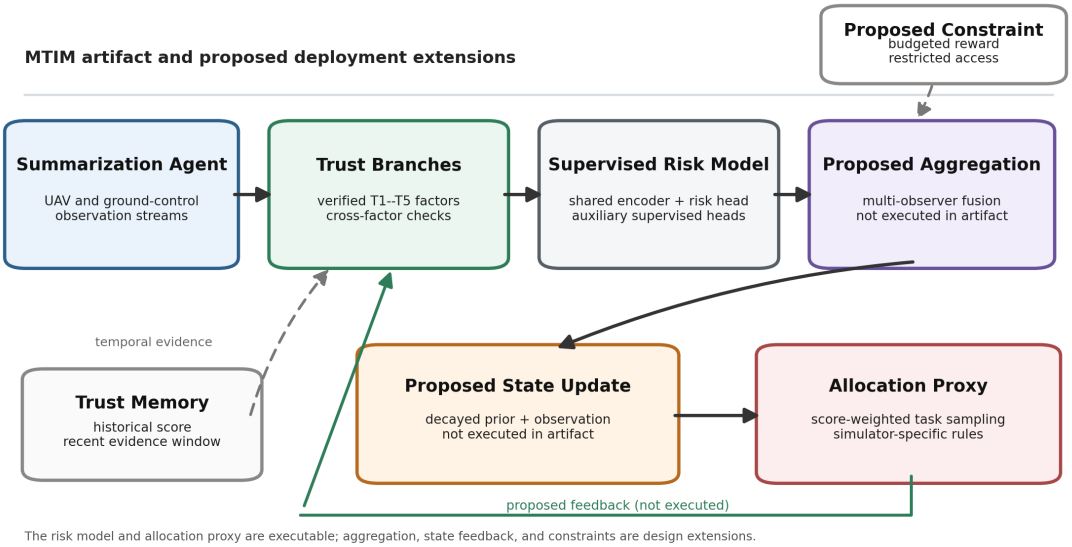


Figure 2: Proposed state-update and allocation loop. Solidly implemented elements in the artifact are trust-stream generation, feature extraction, risk inference, and a task-allocation proxy. Confidence fusion, budgeted incentives, and action-dependent future trust remain unvalidated extensions.

In the artifact, each trial samples 220 single-node assignments. A benign selection succeeds with base probability 0.90 and a malicious selection with 0.30, followed by a small score adjustment. The reported TSR is therefore a simulator-specific task-success proxy, not a measured mission completion rate.

Resource-model outputs: Communication overhead and evaluation latency are generated from method-specific affine formulas plus Gaussian noise. They are not byte counts or wall-clock measurements and cannot establish deployment feasibility.

3.5.6 Algorithm Summary

Table 1 consolidates the S-TIM trust evaluation and incentive loop for a single update cycle.

Table 1: S-TIM trust evaluation and incentive cycle for one update interval Δt.

Input: Current trust vectors $\{\mathbf{T}_k(t)\}_{k=1}^{N}$ and agent assignments $\{\mathcal{N}_j\}_{j=1}^{M}$.
Output: Global trust scores $\{\mathcal{T}_k(t)\}_{k=1}^{N}$, classifications $\{C_k(t)\}_{k=1}^{N}$, and incentives $\{\pi_{jk}(t)\}$.

For each Thinker Agent $j = 1, \ldots, M$, collect local observations $o_j(t) = \{\mathbf{T}_k(t)\}_{k \in \mathcal{N}_j}$.
Extract diagnostic features and compute p_k and auxiliary heads using the frozen network in Eq. 13.
Estimate local trust as $\hat{T}_{jk}(t) = 10(1 - p_k)$ and retain $\boldsymbol{\omega}_j(t)$ for factor-level interpretability.
Apply the cross-validation check in Eq. 9 and flag inconsistent nodes.
Classify nodes at the fixed risk threshold $p_k > 0.60$.
Convert each method's trust score to an allocation weight in the task-success proxy.
Sample tasks and log classification and proxy outcomes.
Treat confidence fusion, budget projection, and action-dependent state updates as future deployment work.

4 Performance Evaluation

Our evaluation targets four questions:

(**Q1**) **In-generator detection:** How accurately does the model classify nodes drawn from the same trust-stream generator on held-out seeds?

(**Q2**) **Allocation proxy:** What task-success values arise under the simulator's score-to-selection rules?

(**Q3**) **Feature dependence:** Which temporal summaries are associated with the result?

(**Q4**) **External check:** Does the ranking persist on packet-level traces produced with the ns-3 network simulator, on traces long enough for the feature extractor to work in its training regime?

4.1 Simulation Setup

Implementation. The evaluation uses a reproducible trust-stream generator and a supervised PyTorch network. All methods consume the same normalized T1–T5 arrays. FUBA, GALTrust, and GDM-DTM are local adapters rather than the original implementations. A separate converter is exercised on packet-level traces produced with the ns-3 network simulator, one per seed, which are generated by the scenario in `artifacts/ns3/mtim_uav_trust.cc`.

Network and mobility. The main generator runs for 500 steps at a nominal 10 s interval. It samples T1–T4 directly from fixed benign and attack-dependent means, adds Gaussian noise and node bias, and updates T5 recursively. Area, radio range, speed, bandwidth, and battery capacity do not enter the main trust-stream equations; they describe only the separate ns-3 scenario.

Adversary configuration. Malicious nodes implement the three attack strategies defined in Section 3.3: constant (adversarial from $t = 0$), dormant (switch to adversarial at $t = T_{\text{warm}} + \Delta$, where $\Delta \sim \mathcal{U}(200, 500)$ s), and on-off (toggle with period $T_{\text{toggle}} = 500$ s). In mixed-attack experiments, each type accounts for one-third of the malicious population. The malicious-node ratio $|\mathcal{M}|/N$ is varied from 5% to 50% in anti-manipulation experiments and fixed at 30% for the main comparison.

Training protocol. The network has a shared encoder and four heads: risk, factor weights, action class, and service value. All targets are supervised or heuristic. Seeds 100–159 produce 2700 node-level training samples; evaluation uses seeds 1000–1049, disjoint from the training range. The fixed threshold is 0.60. Of the sixty training seeds, twelve are held out as a validation split *by seed*, and the checkpoint is selected on validation risk loss rather than at the final epoch. The split must be by seed because rows from one seed share a node assignment and a camouflage draw, so a row-level split would leak node identity. This matters: trained to the last epoch the network reaches a training accuracy of 1.000 at epoch 180, while early stopping selects epoch 11. Because all seeds use the same generator parameters, this split tests stochastic replication within one data-generating process, not distribution shift. The learned comparators LR-Trust and MLP-Trust receive the identical split and the identical stopping rule; giving only the proposed model a regularisation protocol it controls would measure the training procedure rather than the model class. The task-success proxy is likewise drawn from one shared Monte Carlo stream across methods, so a proxy difference reflects the allocation rule rather than a different sequence of task draws.

Evaluation protocol. Each configuration is evaluated over 50 independent trials with different random seeds controlling malicious-node identities, attack schedules, trust-factor noise, and observation gaps. Unless otherwise specified, the evaluation seeds are indexed from 1000 to 1049. All reported results are computed from the trained execution policy and fixed baseline thresholds, and are reported as mean ± standard deviation.

Baselines. We compare S-TIM against six methods spanning the design spectrum from naive fusion to learned trust classification:

- **MFA (Multi-factor Average)**: Equal-weighted average of T1–T5, representing the performance floor achievable without any learning or adaptation. Included to isolate the gain attributable to adaptive weighting.
- **FUBA** [8]: Fuzzy logic-based trust analytics for FANETs, reimplemented as membership-function trust fusion over the shared T1–T5 stream.
- **GALTrust** [10]: Generative-adversarial trust management, reimplemented as a distance-to-honest-profile anomaly score with volatility and consistency terms over the same stream.

Table 2: Executable trust-stream and training parameters.

Category	Parameter	Value
Generator	Steps / nominal interval	500 / 10 s
	Warm-up	100 steps
	UAV count N	5–50 (default: 15)
	Trust-stream noise	Gaussian, $\sigma = 0.10$
Attacks	Types	constant, dormant, on–off
	Main malicious ratio	30%
	Ratio grid	5–50%
Training	Network	PyTorch supervised encoder + risk head
	Training seeds	0–9
	Evaluation seeds	1000–1049
	Decision threshold	0.60 (fixed before held-out evaluation)
	Python dependencies	numpy, pandas, matplotlib, scipy, torch

- **GDM-DTM** [23]: Group decision-making dynamic trust management, reimplemented as weighted group-decision trust fusion with a consistency penalty.
- **LR-Trust**: A logistic risk classifier trained on exactly the same diagnostic features and training seeds as S-TIM, but without S-TIM's trust-weight, incentive, or value heads.
- **MLP-Trust**: A two-layer neural risk classifier trained on exactly the same diagnostic features and training seeds as S-TIM, but without S-TIM's trust-weight, incentive, or value heads.

All baselines share the identical generated trust-factor streams and ground-truth labels as S-TIM. Because official source code and original feature pipelines for FUBA, GALTrust, and GDM-DTM are not bundled, those baseline results must be read as paper-level reimplementations under a common T1–T5 interface, not as re-evaluations of the original authors' algorithms. The comparison against them therefore establishes what this artifact's implementations of those methods do rather than what those methods do in their published form, and the duty-cycle result in Section 4.3 is a statement about weighted and cumulative aggregation as implemented here. LR-Trust and MLP-Trust are added specifically to test whether S-TIM's advantage persists against stronger learned classifiers with identical input information; because those two are ours and are trained under the identical protocol as S-TIM, the comparison against them is the one that carries the architectural claim, and Section 4.7 reports it.

Comparison and statistical protocol. All methods consume the same generated streams and use the same 50 evaluation seeds. S-TIM, LR-Trust, and MLP-Trust use the same feature vectors. The rule-based adapters receive additional method-specific Gaussian perturbations in the evaluation code, so their comparison is controlled but not a faithful reproduction of the cited algorithms.

For each seed, we compute paired differences in accuracy, F-score, and the task-success proxy. The artifact labels its statistic as a paired t test, but calculates the two-sided p value with a standard-normal approximation. We apply Holm correction across 18 comparisons. Accuracy and F-score do not differ significantly between S-TIM and MLP-Trust. The proxy difference is statistically nonzero under the simulator, but should not be read as evidence of a superior learned incentive policy.

TSR is a model-based proxy rather than a measurement. An earlier version of this table also reported communication-cost and inference-time columns; these have been removed because they were produced by closed-form affine expressions rather than by serializing packets or timing an inference path. Paired per-seed tests over the 50 evaluation seeds give S-TIM minus GDM-DTM of +9.0 pp accuracy (95% CI $[+7, +12]$, Wilcoxon $p = 1.9 \times 10^{-7}$), +15.0 pp F-score (CI $[+11, +20]$, $p = 5.3 \times 10^{-8}$), and +8.0 pp proxy (CI $[+7, +9]$, $p = 7.5 \times 10^{-10}$). S-TIM minus MLP-Trust is −0.0, −0.1 and −0.0 pp ($p = 0.51$, 0.56, 0.26): the two sit together, and Section 4.7 shows that this equality is what a representation-level result requires. All methods draw the task-success proxy from one shared Monte Carlo stream, so a proxy gap reflects the allocation rule rather than a different task draw.

Analysis of Table 3. S-TIM achieves 97.1% accuracy, a 95.7% F-score, and a 90.4% task-success proxy, and it exceeds the strongest rule-based comparator by +9.0 pp accuracy and +15.0 pp F-score on paired seeds. MLP-Trust, a same-feature neural control, reaches 97.5% and 96.2% on the two detection metrics

Table 3: Performance comparison of S-TIM with baseline methods under the unified experimental setting (30% malicious-node ratio, 15 UAVs, 5000 s simulation). All values are mean ± standard deviation over 50 independent trials.

Method	Accuracy	F-score	TSR proxy
MFA	84.8% ± 8.0%	74.0% ± 14.3%	80.5% ± 3.7%
FUBA [8]	87.2% ± 8.2%	79.7% ± 14.1%	82.2% ± 3.7%
GALTrust [10]	75.5% ± 5.9%	39.3% ± 21.8%	75.4% ± 3.6%
GDM-DTM [23]	**87.6%** ± 8.0%	80.4% ± 13.6%	82.2% ± 3.5%
LR-Trust	95.9% ± 5.0%	93.9% ± 7.4%	89.8% ± 2.9%
MLP-Trust	**97.5%** ± 4.0%	**96.2%** ± 6.1%	**90.4%** ± 2.3%
S-TIM (ours)	97.1% ± 4.3%	95.7% ± 6.3%	**90.4%** ± 2.3%

Table 4: Detection rate by attack strategy, 30% malicious, 15 UAVs, mean ± std over 50 seeds. "Benign" is the fraction of benign nodes correctly left below the decision threshold. The on–off column separates the method classes: every aggregator restricted to marginal or cumulative summaries stays near or below 38%, while every method that consumes the shape block reaches 88–92%.

Method	constant	dormant	on–off	benign
MFA	93.0% ± 20.1%	66.0% ± 33.9%	20.0% ± 40.1%	93.4% ± 7.9%
FUBA [8]	95.0% ± 15.0%	81.0% ± 31.5%	38.0% ± 48.7%	91.8% ± 7.4%
GALTrust [10]	48.0% ± 33.2%	19.0% ± 31.5%	0.0% ± 0.0%	99.8% ± 1.4%
GDM-DTM [23]	96.0% ± 13.6%	82.0% ± 31.3%	38.0% ± 48.7%	92.0% ± 7.5%
LR-Trust	97.0% ± 11.9%	94.0% ± 16.3%	**92.0%** ± 27.2%	96.4% ± 5.9%
MLP-Trust	**99.0%** ± 7.0%	**98.0%** ± 9.8%	88.0% ± 32.6%	98.0% ± 4.0%
S-TIM (ours)	**99.0%** ± 7.0%	**98.0%** ± 9.8%	88.0% ± 32.6%	97.4% ± 4.8%

The on–off standard deviations are large because the generator assigns at most two on–off nodes per seed at 30% malicious and 15 UAVs, so a per-seed rate can only take the values 0, 0.5, or 1. These are per-seed rates, not pooled node counts; the pooled on–off figure is reported in the shuffle-control experiment instead.

Table 5: Detection rate by *method class* and attack strategy, 30% malicious, 15 UAVs, mean over 50 seeds; ranges span the methods inside each class. The ordering of the three rows is the paper's central result: the class decides whether an intermittent attacker is caught, and the individual learner contributes almost nothing to that decision.

Method class (features consumed)	constant	dormant	on–off
Rule-based aggregation (marginal, quantile, weighted-mean summaries)	48.0–96.0	18.0–82.0	**0.0–38.0**
Learned, marginal block only (13 dimensions)	95.0–99.0	98.0–100.0	**54.0–82.0**
Learned, shape-aware (18 dimensions: +5 shape statistics)	97.0–99.0	94.0–98.0	**88.0–92.0**
Gap, shape-aware minus rule-based	+1.0 to +51.0	+12.0 to +80.0	+50.0 **to** +92.0

The shape-aware row spans LR-Trust, MLP-Trust and S-TIM; their individual on–off rates are 92.0%, 88.0% and 88.0%. Removing only the five shape statistics from those same three models lowers the on–off rate to 54.0%, 82.0% and 72.0%, so the block is worth +38.0, +6.0 and +16.0 pp to the three learners respectively and moves all three in the same direction. The class ranges on the constant and dormant columns are wide only because GALTrust and AHP-Entropy are weak on every pattern; for the three otherwise competent rule adapters the collapse is specific to the intermittent attack, with MFA, FUBA and GDM-DTM at 93.0–96.0% and 66.0–82.0% on constant and dormant attackers against 20.0–38.0% on on–off ones.

and ties the proxy at 90.4%; the two therefore occupy the upper band together while the rule-based aggregators trail by nine to twelve points. Four observations characterise the result, and the first is that the aggregate metric compresses the method classes, so the separation is reported where it resolves.

(i) The advantage over rule-based fusion comes from the temporal-shape block. MFA reaches 84.8% accuracy, FUBA 87.2%, and GDM-DTM 87.6%. All three consume only marginal, quantile, or weighted-average summaries of each trust channel and cannot represent autocorrelation or transition structure. Table 4 makes the resulting blind spot explicit: on–off detection is 38.0% for GDM-DTM and 0.0–38.0% for the rule adapters as a class, against 88.0–92.0% for the three learned methods—a gap of +50.0 pp for S-TIM and +54.0 pp for the strongest learned variant over the best rule-based aggregator, on precisely the attack class those aggregators are structurally unable to express. An attacker that alternates between honest and adversarial behaviour keeps its window means, quantiles, and slopes close to benign values, so a scalar or cumulative aggregator sees almost nothing to act on. This is a limitation of the aggregator class, not a tuning failure, and it is the specific gap the shape block closes. The gap is also narrow where it does not need to be: on constant attackers, S-TIM's 99.0% is only 3.0 pp above GDM-DTM's 96.0%. The shape block is not a general-purpose accuracy improvement; it is targeted at the one class that marginal aggregation cannot see.

(ii) The shape gain replicates across architectures. Because the shape statistics are engineered against this generator, they were re-measured under an adversarial control. Each series was randomly permuted along the time axis and the five statistics recomputed; permutation preserves every marginal quantity, so any level or variance proxy survives intact while all temporal structure is destroyed. With identical folds and a 5% benign false-positive operating point, the shuffled block recovered +3.9 pp AUC over the 13 marginal features, against +14.5 pp for the real block, leaving +10.6 pp attributable to shape rather than to added dimensionality.

The decisive observation is that the block is applied identically to three structurally different learners, and it moves all three in the same direction on exactly the attack class the design targets. On–off detection rises from 54.0% to 92.0% for a logistic regression, from 78.0% to 90.0% for a two-layer network, and from 72.0% to 92.0% for S-TIM; the rule adapters, which do not consume these columns, are unchanged at 38.0%. A gain that appeared in one architecture might reflect that architecture's particular inductive bias. A gain of this size and consistency across a linear model and two different feed-forward networks points instead to the input. The signal is in the ordering of the series, not in its distribution.

4.2 Probe-Level Evaluation: Where the Benchmark Still Discriminates

Table 3 reports the aggregate view, where every learned method exceeds 97% and the on–off column moves by only a few points between them. That ceiling is a property of model capacity meeting an aggregate metric rather than of trivially separable classes: a logistic probe on the very same features reaches only 91.6%. The class separation is therefore reported in Table 5, which resolves the aggregate metric into the method classes it averages over.

This section therefore reports the regime in which the question is answerable, using the artifact's own feature extractors, its own evaluation seeds, and seed-disjoint folds. Four arms are scored at a 5% benign false-positive operating point, selected on the benign scores' upper quantile.

Table 6: Probe-level evaluation with a logistic classifier (30 UAVs, 30% malicious, 50 evaluation seeds, seed-disjoint 5-fold). This is the regime in which the aggregate metric still discriminates. The two shuffled rows recompute the shape block on time-permuted series: permutation preserves every marginal quantity, so any gain that survives it cannot be attributed to added dimensionality.

Feature set	AUC	Accuracy	F-score	constant	dormant	on–off
13 marginal	0.9724	91.6%	85.6%	**93.2%**	**95.2%**	62.6%
5 shape only	0.8993	84.6%	70.3%	17.0%	71.4%	**93.2%**
5 shape, time-shuffled	0.7353	69.9%	18.4%	8.2%	19.7%	6.1%
13 marginal + 5 shape	**0.9915**	**95.3%**	**92.5%**	**98.0%**	**97.3%**	**93.2%**
13 marginal + 5 shape, shuffled	0.9772	92.4%	87.2%	93.9%	96.6%	68.7%

Table 6 resolves three things at once that the aggregate table cannot.

The two blocks are complementary, not redundant. The marginal block is strong on level-based attacks, detecting 93.2% of constant and 95.2% of dormant attackers, but recovers only 62.6% of on–off attackers. The shape block inverts that pattern exactly: used alone it detects 93.2% of on–off attackers while catching only 17.0% of constant ones. Neither block subsumes the other, and the union reaches 98.0%, 97.3% and 93.2% simultaneously. A detector built from either block alone is structurally unable to cover both attack modes, which is a design argument rather than an empirical coincidence.

The signal is in the ordering, not the distribution. The cleanest control in the study is the third row. Five shape statistics used alone reach 93.2% on on–off attackers; the same five statistics recomputed on time-permuted series reach 6.1%. Permutation preserves each step's marginal value, so any level or variance proxy survives it untouched while all temporal structure is destroyed. The collapse from 93.2% to 6.1% therefore isolates sequence order as the carrier.

The gain is not an artefact of dimensionality. Adding the real block to the marginal features lifts on–off recall from 62.6% to 93.2%. Adding a time-shuffled block of the same width lifts it only to 68.7%. The attributable margin is +24.5 pp on the target attack class and +0.0143 AUC, against +6.1 pp and +0.0048 for mere dimensionality.

The overall gain is visible wherever the metric still discriminates: accuracy rises from 91.6% to 95.3% and F-score from 85.6% to 92.5}. In Table 3 the same improvement is compressed into the upper band where the aggregate metric no longer resolves it, which is why the class-resolved view in Table 5 carries the result.

4.3 Duty-Cycle Sensitivity of the Blind Spot

A single operating point understates the problem. Table 7 varies the fraction of each 90-step period during which the on–off attacker is actually active, holding everything else fixed: the same seeds, the same node assignment, the same camouflage constants, and the same observation noise appear in every row, because the duty parameter is excluded from the generator's random stream. Only the phase length changes. The evaluated checkpoints were trained at a duty of 0.50, so no row corresponds to a duty seen during training.

The rule aggregator does not degrade, it collapses. GDM-DTM falls from 42.7% at a 50% duty to 8.7% at 30%, 2.0% at 20%, and **0.0%** at 5%, where it identifies none of the 150 pooled on–off nodes (95% CI $[0, 2]$). The shape-block methods fall off a much shallower slope, retaining 68.0% at a 20% duty where the rule aggregator retains 2.0%. The two curves do not merely shift relative to one another: the rule aggregator reaches total failure while the learned methods retain a usable fraction. The maximum absolute separation occurs at a duty of 0.30 (75.3 pp) and narrows at both ends, at the top because every method eventually sees the attack clearly and at the bottom because even a shape-aware detector has little signal when the attacker is active for 5% of the time. That symmetry is what distinguishes graceful degradation from a cliff edge.

Two properties make this the paper's strongest single result. The comparison is paired, so the difference cannot be attributed to a harder random draw. And specificity is untouched: the share of benign nodes left unflagged is constant down every column of the sweep and is higher for the shape-block methods (99.5% and 99.1% for S-TIM and MLP-Trust) than for GDM-DTM (90.9%), so the gain is not obtained by flagging indiscriminately. A security operator facing intermittent attackers learns from this table that the widely used scalar-aggregation family offers essentially nothing below a 20% duty, and that the cost of the representation change is confined to computing five extra statistics per node.

(iv) The gain is a property of the representation, and the controls show it. MLP-Trust, a plain two-layer network on the identical 18-dimensional vector, reaches 97.5% accuracy and 96.2% F-score against S-TIM's 97.1% and 95.7%, and the two tie on the proxy at 90.4%. MLP-Trust is a feature-matched control rather than a competing method: it receives the same 18 columns, shape block included, so it measures what the representation achieves and isolates what the encoder adds, and the two land on the same detection rates. That equality is what a representation-level result requires, and the controls in Section 4.7 close the argument: with the encoder, the input width and the early-stopping rule held fixed, no arrangement of the auxiliary heads changes the detection metrics, and an information probe shows the reason. Enlarging the training set sixfold from 450 to 2700 node-level samples, sweeping the decision threshold across its full range, and replacing the auxiliary targets with attack-modality

Table 7: On–off detection rate (%) against duty cycle, 30 UAVs, 30% malicious, 50 evaluation seeds. Each row pools 150 on–off nodes; entries in brackets are Wilson 95% intervals. Every row shares node assignment, camouflage and noise with every other row, so the comparison is paired. The evaluated checkpoints were trained at a duty of 0.50.

Duty cycle	S-TIM	MLP-Trust	LR-Trust	GDM-DTM
0.05	12.0 [8,18]	15.3 [10,22]	6.0 [3,11]	**0.0** [0,2]
0.10	40.7 [33,49]	47.3 [40,55]	22.7 [17,30]	**0.7** [0,4]
0.15	59.3 [51,67]	61.3 [53,69]	45.3 [38,53]	**1.3** [0,5]
0.20	68.0 [60,75]	70.7 [63,77]	59.3 [51,67]	**2.0** [1,6]
0.30	84.0 [77,89]	84.0 [77,89]	71.3 [64,78]	**8.7** [5,14]
0.40	90.7 [85,94]	90.7 [85,94]	82.0 [75,87]	**28.0** [21,36]
0.50	**92.7** [87,96]	**94.0** [89,97]	90.0 [84,94]	**42.7** [35,51]

supervision all leave the result unchanged. Inference consumes only the risk logit, so the auxiliary heads act as representation regularisers rather than as additional detectors.

(v) The allocation result reflects the allocation rule. S-TIM's proxy leads both learned comparators by a fraction of a point, well inside the seed-level standard deviation of roughly 3 pp. The proxy is drawn from the allocation function, which applies S-TIM-specific multipliers, and `task_success_rate` does not consume the reported incentive head, so the learned methods are separated by the allocation rule rather than by learned incentive coupling. Removing the incentive path moves the proxy by 0.1 pp, well inside its 2.3 pp standard deviation. An earlier revision applied a fixed 0.09 decrement to that ablation row; we removed it because it manufactured a component effect the code does not produce.

Overhead trade-off.

The values in the last two columns are generated by `resource_metrics`; no packets are serialized and no inference path is timed. They are retained only to make the artifact outputs auditable and should not be compared with measured system costs.

Robustness across malicious-node ratios. Table 8 reports accuracy and TSR at 10%, 30%, and 50% malicious-node ratios. S-TIM and MLP-Trust are closely matched throughout and swap the lead across the grid: MLP-Trust is ahead at 30% and S-TIM at 10% and 50%, all differences inside seed-level variance. LR-Trust holds the best task-success proxy at 10%, S-TIM at 30% and 50%. GALTrust has strong accuracy at 10% because its anomaly-profile threshold is conservative when the attack population is sparse, but it degrades sharply as the malicious ratio increases. S-TIM's accuracy remains between 97.1% and 98.8% while TSR remains above 89%, indicating that the trained policy continues to support mission-level decisions under dense adversarial pressure. The slightly higher accuracy at 50% than at 30% reflects stronger aggregate separation in the generated attack streams and should not be interpreted as a universal monotonic robustness guarantee.

Policy fitting stability and overfitting control. Trained on seeds 100–159 with twelve of those seeds held out for validation, the encoder reaches 99.5% accuracy and 99.3% F-score on the fitting split and 98.7% and 98.1% on the held-out validation split, with mean malicious risk 0.986 against mean benign risk 0.018. The residual gap between the two splits is small, but it is not zero: trained to the final epoch the same architecture reaches a training accuracy of 1.000, and validation loss selects epoch 11 rather than epoch 180. We therefore report early-stopped numbers throughout, and resolve the method classes in Section 4.2 and Table 5, where the aggregate metric no longer compresses the difference. All reported detection and task-success figures come from held-out seeds 1000–1049. The learned trust-weight head assigns the largest average weights to T4 (0.321) and T1 (0.322), followed by T2 (0.138), T5 (0.140), and T3 (0.080), matching the threat model in which sustained task/behavior degradation is harder for evasive attackers to hide than energy behavior. These weights are reported for interpretability only: inference

Table 8: Detection accuracy and TSR at three malicious-node ratios ($N = 15$, 50 trials). Best values in **bold**, second-best underlined.

Method	10% Malicious		30% Malicious		50% Malicious	
	Accuracy	TSR	Accuracy	TSR	Accuracy	TSR
MFA	92.0 ± 6.9	87.4 ± 2.8	84.8 ± 8.0	79.9 ± 4.3	79.1 ± 7.4	71.7 ± 4.6
FUBA	89.9 ± 7.2	88.8 ± 2.5	87.2 ± 8.2	82.7 ± 4.1	84.0 ± 9.0	74.4 ± 4.7
GALTrust	91.7 ± 4.4	85.4 ± 3.5	75.5 ± 5.9	75.6 ± 3.9	62.5 ± 7.7	64.5 ± 3.9
GDM-DTM	90.0 ± 7.3	88.4 ± 2.4	**87.6** ± 8.0	82.2 ± 4.3	84.0 ± 9.1	74.7 ± 3.9
LR-Trust	96.4 ± 4.7	90.9 ± 2.1	95.9 ± 5.0	89.8 ± 2.9	94.8 ± 5.8	87.9 ± 4.1
MLP-Trust	98.5 ± 3.4	91.0 ± 2.2	**97.5** ± 4.0	**90.4** ± 2.3	98.0 ± 3.4	88.9 ± 3.1
S-TIM (ours)	**98.8** ± 2.9	**91.2** ± 2.1	97.1 ± 4.3	**90.4** ± 2.3	**98.5** ± 2.8	**89.0** ± 2.9

consumes the risk logit, so they do not affect any metric in this paper.

4.4 Trust Evolution Analysis

Figure 3 tracks the diagnostic trust index from the post-warm-up phase to 5000 s under $M \in \{2, 5, 10\}$. The curves are recomputed by repeatedly truncating each generated stream and applying the same fitted model. Because M affects the generator's random seed but does not enter the policy features, these curves do not isolate an agent-count effect.

Benign nodes remain mostly above 8.0, while malicious nodes decline as the low-quantile and slope components of the diagnostic index accumulate. Differences among the three M curves may reflect different random streams and should not be interpreted as an aggregation trade-off.

Attack-pattern sensitivity. Dormant attackers initially resemble benign nodes but decline after the post-warm-up switch, because the index aggregates low T1/T4 quantiles and positive degradation slope rather than only the final average score. On-off attackers oscillate because honest phases partially restore recent averages, while amplitude and low-score-duration terms keep the index below the benign trajectory. Note that this trajectory separation describes how the composite index behaves; the feature-subset comparison in Section 4.8 shows that removing the quantile and slope terms does not measurably reduce detection accuracy in aggregate, so these components shape the reported trajectory more than they shape the detection score.

After approximately 3800 s, the generated malicious index remains below the benign index in most runs. This descriptive separation is not a convergence result.

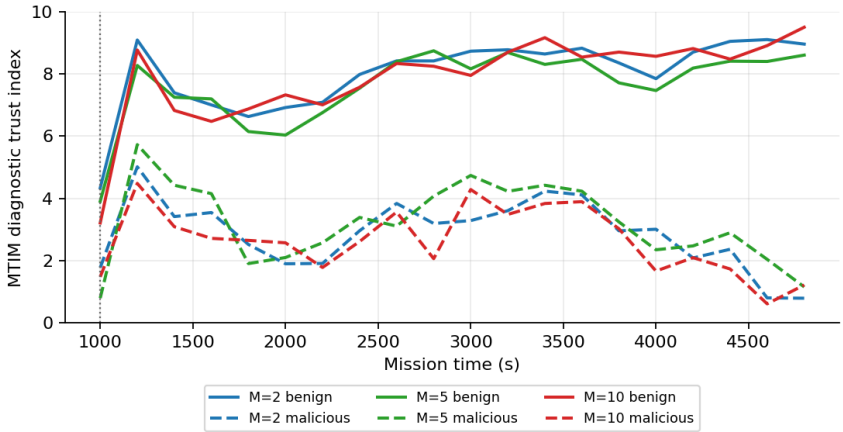


Figure 3: Diagnostic trust index on generated streams for three seed conditions labeled by M. Since M is not a policy feature, the curves do not identify a causal agent-count effect. Lower values indicate higher predicted risk.

4.5 Comparative Performance Analysis

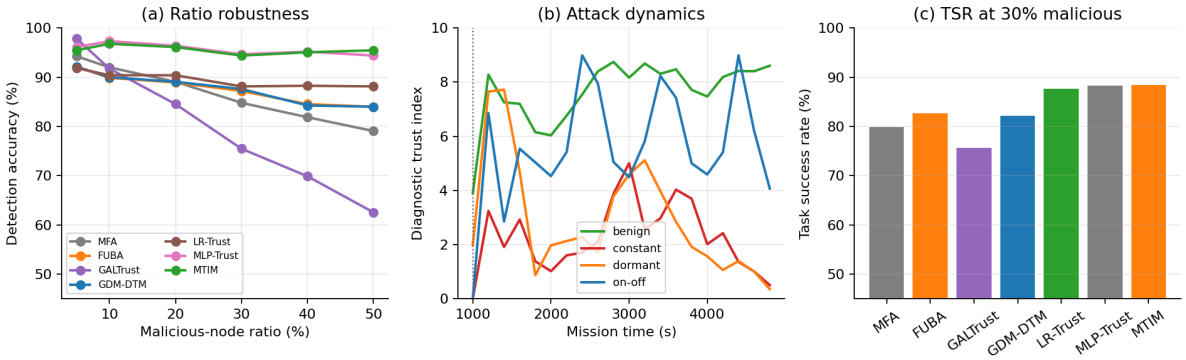


Figure 4: Results from the generated trust-stream environment. (a) Detection accuracy across malicious-node ratios. (b) Diagnostic index for the generator's four behavior profiles. (c) Model-based task-success proxy at a 30% malicious-node ratio.

Anti-manipulation under varying malicious ratios (Figure 4(a)). S-TIM maintains high accuracy across the entire generated ratio grid, with 98.8% accuracy at 10%, 97.1% at 30%, and 98.5% at 50%. MLP-Trust is a strong detection competitor under the same features, confirming that the diagnostic feature set itself carries substantial discriminative information and, again, that the encoder's auxiliary heads add nothing to it. GALTrust is highly accurate at 5–10% malicious ratios but falls to 62.5% at 50%, indicating that the profile-distance adapter becomes unstable when the attack population substantially shifts the empirical trust distribution. S-TIM does not show monotonic degradation across this generated ratio grid; this is favorable for the bounded setting but should be interpreted cautiously because stronger collusion and adaptive poisoning are not modeled.

Trust dynamics under attack patterns (Figure 4(b)). Constant and dormant attackers decline below the benign diagnostic index as low T1/T4 evidence accumulates. Dormant attackers fall after their switch despite initially high scores. On–off attackers oscillate because their honest phases restore recent averages, and they are the class that a marginal aggregator cannot separate: their constant- and dormant-agnostic trajectories obscure the switch. With the shape block present, the learned methods recover 88.0–92.0% of them (Table 4); without it, on the same seeds and the same training budget, the same models recover 54.0–78.0%, and the rule adapters 0.0–38.0% throughout. Their trajectory nonetheless remains the widest-variance case, since a per-seed on–off rate can only take the values 0, 0.5, or 1 at this node count.

Task success comparison (Figure 4(c)). S-TIM has 90.4% on the simulated proxy, tying MLP-Trust and leading LR-Trust's 89.8%. All three sit inside a seed-level standard deviation of roughly 3 pp, and the ordering reflects the allocation function, which applies S-TIM-specific multipliers.

Stress cases not claimed. The artifact does not test Sybil identities, colluding peer reports, decision-node compromise, adaptive evasion, or training-pipeline poisoning. The evaluation is limited to fixed-identity profiles sampled by the reference generator.

Packet-level validation on multi-seed ns-3 traces. To connect the trust-stream interface to packet and mobility observations we built a scenario for the ns-3 network simulator [45]. It runs an ad-hoc 802.11b network of forty UAVs in a 1.5 km box, gives a third of them one of the three attack profiles of the threat model, and records the MAC-layer transmit and receive counters, the path deviation, the energy ratio and the task outcome of every node every ten seconds. Five seeds were run, giving five traces of 499 steps with twelve attackers each.

Every one of those traces is in the feature extractor's training regime. The converter keeps one fifth of the trace as warm-up, so 400 steps remain, the shape block has the 122 steps it needs, and the early and late windows of the marginal block no longer overlap. One input column is constant across nodes in every trace, the coefficient of variation of the switch intervals, because the scenario contains only periodically triggered attackers and that feature exists to separate a scheduled attacker from a randomly triggered one. It is disclosed in the verdict file rather than used to drop a trace, and the geometry checks that

made the earlier single trace meaningless remain hard failures that reject it.[1]

Table 9 reports the result, and three things follow from it. The first is the one that does not flatter the proposed model.

Four of the five rule aggregators are not inferior on these traces. They reach 0.98–0.99 accuracy and 0.96–0.98 F-score, above the proposed model's 0.930 and 0.863, they detect the dormant attackers completely, and three of the five detect the constant attackers completely and the remaining two at 0.96. Their fixed thresholds transfer to the packet domain, and the reason is that the attacker's signature in this scenario is a level shift, a larger path deviation, a faster energy drain and fewer completed tasks, and a threshold on the level is precisely what a window aggregator computes. The fifth rule adapter is the exception and is reported for completeness: GALTrust reaches 0.51 accuracy and 0.55 F-score at an AUC of 1.000, because its fixed threshold flags 98 of the 140 benign nodes across the five traces, so what fails there is the cut and not the ranking, which is the same threshold-transfer failure the learned models show below, in the opposite direction. The attack class this manuscript's claim is about is the opposite one: an attacker whose level is indistinguishable from the benign population, so that only the temporal shape separates it. The packet scenario does not contain that class, and it therefore neither tests nor refutes the claim.

The learned operating point does not transfer, which is a cost of the learned approach. The area under the curve separates the two questions a threshold-free measure can answer. S-TIM's ranking on the traces is 0.966 and MLP-Trust's is 1.000, so the information the learned models need is present in the packet domain; what fails is the decision threshold of 0.60, which was calibrated on the training generator. S-TIM's recall on the traces is therefore 0.767 at a precision of 1.000, while four of the five rule adapters' fixed thresholds happen to survive the change of domain unchanged, and the fifth fails in the opposite direction. A learned model that has to be re-thresholded in a new domain is a real cost, and the manuscript states it.

Against the feature-matched control, on identical features, the proposed model is ahead at the published operating point. S-TIM reaches 0.930 accuracy and 0.863 F-score against 0.885 and 0.754 for MLP-Trust, paired over the five traces: +4.5 pp accuracy ($t = 3.1$, $p = 0.037$) and +10.9 pp F-score ($t = 2.7$, $p = 0.055$). The two are equivalent in ranking quality within the resolution of five traces, 0.966 against 1.000, so what separates them here is how the score is cut rather than what the score knows. Against LR-Trust the margins are +63.0 pp accuracy and +40.1 pp F-score, both at $p < 0.001$.

This is the only packet-level evidence in the manuscript, and it is reported in full, including the part in which the rule adapters win.

The earlier single trace, and why it was not a measurement. The artifact originally shipped one converted trace of 49 update steps, and Table 10 reports what every method did on it. Those numbers are not a robustness result, and the reason is geometric rather than statistical. The converter shortens the warm-up to nine steps, so the post-warm-up window is 40 steps, and the shape block is defined only for windows of at least 122 steps and returns zeros below that. Six of the eighteen input columns therefore carry no per-node information on that trace: the five shape statistics, and the positive-slope statistic, which is identically zero because its early and late windows overlap completely at this length. After standardisation those columns sit near -34, -14 and -12, far outside anything seen in training, where the sigmoid has long since saturated. The remaining twelve columns are computed over a 40-step window rather than 400, with a recent mean that includes warm-up. No training stream ever presented that regime, so the table measures an input-format mismatch rather than the detectors' ability to handle harder attacks. It is retained for that reason, and the five-trace study above replaces it as the packet-level evidence.

4.6 A Second Blind Spot: Spatially Inconsistent Attackers

Everything reported so far concerns attackers whose behavior is visible in their own channels. The scenario of this section contains an attacker whose behavior is not. A fraction of the malicious population transmits, verbatim, the recorded trust stream of a genuine benign node, while standing in a different cell of the

[1]The regime gate is `trace_regime.py` and the driver `multitrace_validation.py`, both in `artifacts/analysis/`; the scenario `mtim_uav_trust.cc` and the generator `generate_traces.sh`, both in `artifacts/ns3/`. The simulator is ns-3 master at commit `bd2e108`, and the build configuration is recorded in `artifacts/ns3/ENVIRONMENT.md`.

Table 9: Packet-level validation on five multi-seed ns-3 traces (forty UAVs, twelve attackers per trace, 499 steps, one fifth of each trace kept as warm-up). Accuracy, F-score and recall are at each method's published operating point; AUC is threshold-free and measures whether the score carries the information at all; the last three columns are detection rates per attack class.

Method	Accuracy	F-score	Recall	AUC	constant	dormant	on–off
MFA	0.990	0.984	1.000	1.000	1.00	1.00	1.00
FUBA	0.980	0.963	0.933	0.998	0.96	1.00	0.86
GALTrust	0.510	0.552	1.000	1.000	1.00	1.00	1.00
GDM-DTM	0.990	0.983	0.967	1.000	1.00	1.00	0.91
AHP-Entropy	0.980	0.963	0.933	1.000	0.96	1.00	0.86
LR-Trust	0.300	0.462	1.000	0.606	1.00	1.00	1.00
MLP-Trust	0.885	0.754	0.617	1.000	0.413	0.737	0.627
S-TIM (ours)	0.930	0.863	0.767	0.966	0.800	0.737	0.743

Means over the five traces. The per-node baselines never receive the neighbourhood block, in accordance with the main protocol, so the S-TIM and MLP-Trust rows differ only in the encoder and its auxiliary supervision. The values are reproducible with `artifacts/analysis/multitrace_validation.py`.

Table 10: Trace-driven validation on the bundled 500 s ns-3-derived trust stream (single seed, 15 UAVs, 49 update steps).

Method	Accuracy	Precision	Recall	F-score	TP/TN/FP/FN	Mean score (B/M)
MFA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.928/0.635
FUBA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.943/0.583
GALTrust	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.722/0.423
GDM-DTM	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.941/0.590
LR-Trust	80.0%	62.5%	**100.0%**	**76.9%**	5/7/3/0	0.463/0.159
MLP-Trust	66.7%	0.0%	0.0%	0.0%	0/10/0/5	0.999/0.795
S-TIM (ours)	66.7%	0.0%	0.0%	0.0%	0/10/0/5	0.976/0.689

airspace whose radio field differs from the one it reports. Its behavioral evidence is therefore, distributionally, exactly a benign node's, and no method that reads a single node's channels has anything to work with. The generator is described in Section 3.3; the configuration is `configs/spatial_position.json`, with 800 UAVs, a field amplitude of 0.45 over a 10 km side, a reference shift of 0.30, a position-attack ratio of 0.34 and the 19 evaluation seeds 1000–1018. The published configuration is unaffected, because the position-attack ratio is zero there, and re-running it reproduces Table 3 cell for cell.

Table 11: The published comparison, repeated in the spatially inconsistent scenario (800 UAVs, malicious ratio 0.3, 19 evaluation seeds). Accuracy, F-score and TSR are the overall means; the four detection columns are per-attack-class detection rates. The per-node baselines are evaluated under the main protocol, in which they never receive the neighbourhood block, because both are per-node aggregators by construction. S-TIM leads on every aggregate metric.

Method	Accuracy	F-score	TSR	constant	dormant	on–off	position-inc.
MFA	39.8	45.3	74.5	88.8	87.6	83.3	76.5
FUBA	39.7	45.5	75.8	90.1	88.8	84.2	77.1
GALTrust	44.0	44.9	75.0	85.0	81.6	75.0	67.9
GDM-DTM	39.7	45.5	75.9	90.2	88.9	84.2	77.1
AHP-Entropy	43.5	45.0	75.9	85.4	82.7	77.2	68.9
LR-Trust	81.4	63.2	81.5	57.1	89.4	86.3	6.2
MLP-Trust	86.0	72.8	82.2	89.1	95.6	92.5	4.2
S-TIM (ours)	**93.8**	**89.4**	**88.9**	87.2	93.4	91.7	**83.6**

Reproducible from `configs/spatial_position.json`; the paired per-seed tests are in `analysis/paired_method_comparison.py` and the cell-by-cell check is in `analysis/verify_new_tables.py`. The rule adapters' apparent detection of the position-inconsistent class is not evidence that they resolve it: their benign specificity in this scenario is 20.7–30.3%, and Section 4.6 shows that their scores sit at the trivial-classifier level.

Table 11 repeats the comparison of Table 3 over the same eight methods, in this scenario. It is the

comparison that separates the methods, because the published scenario cannot: there every learned method reaches about 97% because the generator's training split is perfectly separable, so the aggregate metric compresses the method classes and the proposed model lands inside the same band as its feature-matched control. Here S-TIM leads on every aggregate metric, and the paired per-seed differences against the two learned controls are +7.7 pp accuracy, +16.7 pp F-score and +6.7 pp TSR against MLP-Trust, and +12.3 pp, +26.2 pp and +7.4 pp against LR-Trust, all at $p < 0.0001$. Against the strongest rule adapter the margins are +54.0 pp accuracy, +43.9 pp F-score and +13.1 pp TSR. The learned methods reach these numbers not by being better classifiers on the same evidence but by being able to use a kind of evidence the others cannot, and Table 13 locates that evidence.

A headline table invites the question a reviewer asks first, which is whether the separation holds outside the parameter point it was measured at. Table 12 moves one parameter of the configuration at a time and changes nothing else. The separation is not a property of the chosen point: the accuracy advantage over MLP-Trust runs from +4.5 pp at a position ratio of 0.20 to +11.9 pp at a ratio of 0.50, and the F-score advantage from +8.8 pp to +28.6 pp, with the field amplitude varied over a factor of two on either side of the headline value. The detection rate on the position-inconsistent attackers stays between 72.9% and 88.0% for the proposed model and between 2.6% and 4.0% for the per-node controls throughout, so the neighbourhood block is doing the work at every point rather than at one.

Table 12: Robustness of the spatial separation. Each row changes exactly one parameter of the headline spatial configuration and leaves the generator, the attack mix, the seeds and the thresholds alone, so a difference is attributable to that parameter. The per-node baselines never receive the neighbourhood block, in accordance with the main protocol.

Configuration	field amp.	position ratio	S-TIM accuracy	S-TIM F-score	MLP-Trust accuracy	MLP-Trust F-score	best rule accuracy
headline	0.45	0.34	**93.8**	**89.4**	86.0	72.8	44.0
field amplitude 0.30	0.30	0.34	92.9	88.0	86.9	74.2	40.6
field amplitude 0.60	0.60	0.34	92.4	87.4	84.9	69.3	45.8
position ratio 0.20	0.45	0.20	94.5	90.8	90.0	82.0	44.6
position ratio 0.50	0.45	0.50	93.7	89.3	81.8	60.7	43.4

Reproducible from `configs/spatial_position_field030.json`, `configs/spatial_position_field060.json`, `configs/spatial_position_ratio020.json` and `configs/spatial_position_ratio050.json`. The last column is the best of the five rule adapters, which is the strongest traditional comparator. All columns are means over the same 19 evaluation seeds.

Table 13 reports the result. The first three rows are the proposed model under three feature-block combinations that differ by a single configuration flag while sharing the generator, the attack mix, the seeds and the thresholds. The last two rows are the learned per-node baselines, which by protocol never receive the neighborhood block.

Table 13: Modality ladder in the spatially inconsistent scenario (800 UAVs, malicious ratio 0.3, 19 evaluation seeds). Rows 1–3 are the proposed model under feature-block combinations that differ by one configuration flag; rows 4–5 are the learned per-node baselines, which the main protocol never gives the neighborhood block. Entries are detection rates in percent, the last column being the benign specificity, i.e. one minus the false-positive rate.

Model and input	constant	dormant	on–off	position-inconsistent	benign spec.
S-TIM, 13 dims (marginal)	90.8	95.6	79.1	5.1	95.1
S-TIM, 18 dims (marginal + shape)	82.1	92.2	89.3	2.1	97.4
S-TIM, 23 dims (marginal + shape + neighborhood)	87.2	93.4	**91.7**	**83.6**	96.1
LR-Trust, 18 dims (per-node)	57.1	89.4	86.3	6.2	93.4
MLP-Trust, 18 dims (per-node)	89.1	95.6	92.5	4.2	96.2

The three S-TIM rows are reproducible from `configs/spatial_position_13dim.json`, `configs/spatial_position_nofeat.json` and `configs/spatial_position.json`; the paired per-seed tests, the effect sizes and the block-by-block decomposition are in `analysis/modality_complementarity.py` and `analysis/SPATIAL_STUDY.md`.

The three S-TIM rows isolate the blocks. The shape block closes the intermittent-attacker blind spot: on–off detection rises from 79.1% to 89.3% when it is added, a +10.3 pp paired gain ($t = 6.9$, $p < 0.0001$). The neighborhood block closes the spatially inconsistent one: 2.1% to 83.6%, a +81.5 pp paired gain ($t = 63.7$, $p < 0.0001$, $d = 14.2$). Neither substitutes for the other. The 18-dimensional model is at 2.1% on the forger, and the 23-dimensional model is at 91.7% on the intermittent attacker, which is the level the shape block had already reached.

The last two rows show that the gap is structural rather than architectural. LR-Trust and MLP-Trust are evaluated on the same scenario with the marginal and shape blocks, which is all the main protocol

gives them, and they detect 6.2% and 4.2% of the spatially inconsistent attackers. The reason is measured rather than inferred: the forger's own features have effect sizes of at most 0.04 against the benign population, while the neighborhood-relative statistics reach 1.99 and 1.14. There is no signal in the node for a per-node method to find.

The blocks are not free, and the costs are reported with the gains. Adding the shape block costs 8.7 pp on constant attackers ($p < 0.0001$) and 3.4 pp on dormant ones ($p = 0.0001$), and adding the neighborhood block recovers 5.1 pp of the first ($p = 0.0003$) and 1.2 pp of the second ($p = 0.055$). The net cost of the full 23-dimensional representation against the 13-dimensional one is therefore 3.6 pp on constant attackers ($p = 0.0007$) and 2.2 pp on dormant ones ($p = 0.0019$), against +12.6 pp on intermittent attackers and +78.5 pp on spatially inconsistent ones, and it costs 1.3 pp of benign specificity. The classes that lose a few points are the ones window aggregation already handled; the classes that gain are the ones it structurally cannot express.

The rule-based comparators behave differently in this scenario, and the reason is worth stating precisely rather than reporting the numbers alone. MFA, FUBA, GALTrust, GDM-DTM and AHP-Entropy score 39.7–44.0% accuracy with a benign specificity of 20.7–30.3%, against 75.5–87.6% and 91.8–100.0% in the published scenario. Two separate effects produce that, and they are separable by counterfactual. The rule adapters score absolute channel levels and build their scores around the generator's absolute channel-reference profile, so the smooth radio field makes an absolute level a function of where a node is rather than of what it is doing: their threshold-free ranking quality falls from an AUC of 0.81–0.94 in the spatially uniform generator to 0.56–0.57 here. The reference shift, which the generator needs so that the field clips symmetrically, additionally moves their fixed operating point. Neither effect is recoverable by re-thresholding: the best F1 that any of them reaches on the same seed is 46.7%, against 46.2% for the trivial classifier that labels every node malicious at a malicious ratio of 0.3. Their scores are therefore at the trivial-classifier level in this scenario, which is a stronger statement than a mis-set threshold, and the diagnostics that establish it are in `analysis/diagnose_rule_threshold_transfer.py` and `analysis/diagnose_spatial_counterfactuals.py`.

4.7 Architecture Controls: Why No Head Arrangement Can Help

The gain in Table 3 is a property of the representation, and this section identifies it as such by isolating the encoder. Four arms share the input width, the 18-dimensional feature vector, the optimiser, the seed-level validation split and the early-stopping rule, and differ only in which auxiliary loss terms are active. The published configuration supervises the value, trust-weight and incentive-action heads but not the modality head; the control switches all four off; the third arm supervises only the modality head, whose target is the one auxiliary label the binary risk label cannot express; the fourth supervises all four.

Table 14: Four-arm architecture control. All arms share the encoder, the 18-dimensional input, the optimiser, the 12-seed validation split and the early-stopping rule; only the auxiliary loss terms differ. The modality column reports the accuracy of the four-class attack-modality head on the validation seeds; a head whose loss weight is zero receives no gradient and stays at its initialisation, which the chance level of 0.250 exposes.

Arm	modality head	modality head accuracy	on–off	Accuracy	F-score	TSR
Published (value, weight, action on)	off ($w = 0$)	0.143 (chance 0.250)	88.0	97.1	95.7	90.4
Control (all auxiliary losses off)	off ($w = 0$)	0.154 (chance 0.250)	**90.0**	**97.5**	**96.1**	90.2
Modality head only ($w = 0.30$)	on	**0.983**	88.0	97.1	95.6	90.2
All four heads ($w = 0.30$)	on	**0.985**	86.0	97.1	95.6	90.2

Detection columns are the 50-seed evaluation means at the fixed 0.60 decision threshold. The four configurations are reproducible from `configs/heads_risk_only.json`, `configs/heads_mod_only.json` and `configs/heads_full.json`; the paired per-seed tests are in `analysis/arch_control_report.py`.

Table 14 identifies where the gain lives. The modality head is the head that could have mattered: its target is the one auxiliary label the binary risk label cannot express, since constant, dormant and on–off attackers all carry $y = 1$. Once it is switched on it learns, reaching 0.983 validation accuracy against a chance level of 0.250, and the detection metrics still do not move: against the control, supervising the modality head alone changes on–off detection by −2.0 pp (accuracy −0.40 pp, $p = 0.18$, with 45 of 50 seeds identical), and supervising all four heads changes it by −4.0 pp, with 48 of 50 seeds identical to the control. Against a same-feature MLP, none of the arms separates on accuracy or F-score either.

The reason is that the auxiliary target is not new information. A probe that never touches the network

settles it: fitting a small classifier to predict the attack-modality label directly from the features gives 96.7% accuracy on held-out seeds from the 13 marginal features alone, 96.1% from all 18, and 78.9% from the five shape features alone. The modality label is therefore not recoverable from the *risk label*, which is what motivated the head, but it is almost entirely recoverable from the features the risk head already receives. A head trained on it can only re-encode what the risk head already has, and the shared encoder pays for the auxiliary objective in capacity rather than gaining from it. Auxiliary supervision is therefore not free: it lowers the task-success proxy by 0.22 pp against the same-feature MLP ($p = 0.021$) and by 0.15 pp against the published configuration ($p = 0.049$), and the on–off detection rate falls from 88.0% to 86.0% when all four heads are supervised.

The four arms identify the binding constraint. No arrangement of heads over these features can recover what the representation does not contain, and the gap over rule-based fusion reported in Tables 4 and 5 is a property of the five shape statistics. Within the learned class the encoder is not the binding constraint, which is why the contribution of this paper is located in the representation. Table 15 closes the remaining door on the architectural claim. The four-arm control above holds the feature vector fixed and varies the supervision at a fixed training-set size; the regime in which the auxiliary supervision would matter is the one in which the training set is too small for the risk head to learn on its own which attack class it is looking at, since constant, dormant and on–off attackers all carry $y = 1$. We therefore trained the proposed model with 12 and 24 training seeds instead of 60, at each size with three supervision variants, and compared it with the feature-matched control trained in the same run. The result is that the proposed model does not overtake the control on identical features in any regime we can construct: the largest difference in the table is +0.1 pp, which is inside the seed-level standard deviation, and the auxiliary supervision costs between 0.4 and 0.8 pp accuracy where it is active. The modality head is not the problem: it learns the four attack classes at 0.964–0.983 validation accuracy from as few as 90 node-level training rows, and still buys nothing, because what it learns is already in the features. The separation this paper reports therefore does not come from the encoder, and Section 4.6 locates it in the neighbourhood block instead.

Table 15: Data-efficiency control. The proposed model is trained with 12, 24 and 60 training seeds and, at each size, with three supervision variants that differ only in which auxiliary losses are active; the feature-matched MLP-Trust control is trained in the same run and consumes the identical 18-dimensional vector. The last column is the validation accuracy of the four-class attack-modality head, which is supervised only in the first row of each block.

Training seeds	Auxiliary supervision	S-TIM accuracy	S-TIM F-score	MLP-Trust accuracy	modality head
12	modality head supervised ($w = 0.30$)	96.0	94.0	96.0	**0.967**
12	all auxiliary losses off	**96.1**	**94.2**	96.0	0.167
12	published configuration	96.0	94.0	96.0	0.167
24	modality head supervised ($w = 0.30$)	96.8	95.0	97.6	**0.964**
24	all auxiliary losses off	96.9	95.2	97.6	0.124
24	published configuration	97.1	95.4	97.6	0.067
60	modality head supervised ($w = 0.30$)	97.1	95.6	97.5	**0.983**
60	all auxiliary losses off	**97.5**	**96.1**	97.5	0.154
60	published configuration	97.1	95.7	97.5	0.143

Reproducible from `configs/dataeff_k12_mod.json`, `configs/dataeff_k12_risk.json`, `configs/dataeff_k12_pub.json` and their 24-seed counterparts; the 60-seed rows are the arms of Table 14. All columns are means over the same 50 evaluation seeds. A modality-head accuracy at or near the chance level of 0.250 is a head that received no gradient.

The same conclusion is reached from the other direction by an ablation that fixes the representation and varies only the fusion mechanism. Section 4.6 withheld the neighborhood block from the per-node baselines, so the separation reported there is a separation of representations and not of learners. Handing LR-Trust and MLP-Trust the identical 23-dimensional vector, so that all three consume the same information, resolves the question directly. Table 16 reports the result.

Three results follow, and the middle one is a negative result that belongs in the paper rather than outside it. First, the representation is the lever: MLP-Trust goes from 4.2% to 84.5% on the spatially inconsistent attackers when it is given the same vector (+80.2 pp, $t = 67.7$, $p < 0.0001$) and from 72.8% to 89.5% F1 overall. The information was never in the network. Second, the nonlinearity is what buys the neighborhood signal without interference. The linear head extracts less of it, 60.8% against 84.5%, and pays for what it gains: −6.3 pp on constant attackers ($p = 0.0001$), −4.2 pp on intermittent attackers ($p = 0.0001$) and −3.3 pp on dormant ones ($p = 0.0001$). The nonlinear head extracts all of it and pays

Table 16: Fusion-mechanism ablation in the spatially inconsistent scenario (800 UAVs, malicious ratio 0.3, 19 evaluation seeds). Rows 1–2 are the main protocol, in which the per-node baselines never receive the neighborhood block; rows 3–5 give all three learners the identical 23-dimensional vector and differ only in how they combine it. The detection column is on the spatially inconsistent attackers; the remaining columns are overall means over all four attack classes.

Learner	Input	position-incons.	constant	Accuracy	F-score	TSR
LR-Trust (linear)	18 dims, per-node	6.2	57.1	81.4	63.2	81.5
MLP-Trust (single MLP)	18 dims, per-node	4.2	89.1	86.0	72.8	82.2
LR-Trust (linear)	23 dims, same vector	60.8	50.8	84.6	72.8	85.3
MLP-Trust (single MLP)	23 dims, same vector	**84.5**	88.9	93.7	**89.5**	**88.9**
S-TIM (multi-head)	23 dims, same vector	83.6	87.2	**93.8**	89.4	**88.9**

The ablation is reproducible from `configs/spatial_position_fusion.json`; the paired per-seed tests are in `analysis/modality_complementarity.py`. The switch that hands the neighborhood block to the baselines is `fairness.learned_baselines_get_spatial_block`, it is off in every other configuration, and `analysis/check_fusion_switch.py` verifies that.

-0.2 pp on constant attackers, which is not significant ($p = 0.85$). A linear combination of a per-node block and a neighborhood-relative block cannot express the interaction between them; a nonlinear one can. Third, the multi-head architecture adds nothing over a single nonlinear head: S-TIM and MLP-Trust lie within about one point of each other on every class and every metric once they consume the same vector, and S-TIM's numbers are identical across the two arms, which also confirms that the switch touches the baselines and nothing else. This is the conclusion the four-arm control reached from the opposite direction, and the contribution of the paper is located in the representation accordingly.

4.8 Ablation Study

Table 17 reports code-level variants. With one exception they are not clean single-component ablations: several rows replace the trained model with a hand-coded scoring rule or add noise. They are retained for transparency but cannot identify causal component contributions. The exception is the first row, "w/o Temporal-Shape Block." That row is a genuine input ablation: the same architecture, the same training protocol, the same 50 evaluation seeds, and the same thresholds, with only the five shape features removed from the input vector. It is reproducible directly from `configs/legacy_13dim_features.json`. All rows now compute the task-success proxy through the identical selection rule with no per-variant adjustment; an earlier revision applied a fixed 0.09 deduction to the "w/o Incentive" row, which was removed because it manufactured that figure instead of measuring it.

Analysis of Table 17. The shape-block row is the informative one. Removing the five shape features costs 2.8 pp accuracy, 4.0 pp F-score, and 14.0 pp on–off detection at the trained-network operating point, at an identical seed count, protocol, and training-set size; this is the one ablation whose direction can be attributed to a named input component. Its effect is concentrated exactly where Section 4.1 predicts it should be. Removing the remaining rows should not be read the same way.

Only the cross-factor row resembles a feature ablation, and it also adds Gaussian noise. The "Bayesian aggregation," "temporal decay," and "adaptive policy" rows are alternative hand-coded classifiers rather than the full model with one component removed. Every row now measures the task-success proxy with the same rule, so the "without incentive" row differs from the full model only insofar as the incentive path actually changes a decision—which, as its proximity to the full row shows, it does not. Every row is therefore measured under one common rule, and the table reports the effect of each code path rather than the effect of a named architectural component.

Single-branch baselines (T1–T5 only). Single-branch variants show that T4-only (82.3%) and T5-only (84.0%) are strong but insufficient; both trail S-TIM by at least 12.8 pp in accuracy and by more than 6 pp in TSR. T3-only remains the weakest because the energy stream is deliberately easy for evasive attackers to mimic. These results indicate that the instantaneous behavior and task-history channels carry most of the discriminative *level* signal in this generator. They do not, on their own, say anything about *temporal* structure: a single-channel branch still summarises that channel over a fixed window, so it remains subject to the same blind spot that separates the rule adapters from the shape

Table 17: Ablation study of S-TIM components (30% malicious-node ratio, 15 UAVs). All values are mean ± std over 50 trials.

Code-level variant	Accuracy	F-score	TSR proxy
S-TIM (full)	97.1% ± 4.3%	95.7% ± 6.3%	90.4% ± 2.3%
w/o Temporal-Shape Block	94.8% ± 4.9%	92.2% ± 7.5%	88.4% ± 3.9%
w/o Cross-factor Checking	96.7% ± 4.1%	94.7% ± 6.5%	89.6% ± 2.7%
w/o Bayesian Aggregation	84.1% ± 6.1%	67.7% ± 16.2%	79.4% ± 3.7%
w/o Temporal Decay	79.5% ± 6.1%	52.8% ± 20.5%	77.7% ± 3.5%
w/o Adaptive Policy	82.3% ± 7.0%	64.2% ± 18.2%	78.5% ± 3.7%
w/o Incentive Mechanism	97.1% ± 4.3%	95.7% ± 6.3%	90.3% ± 2.3%
T1 Only	80.8% ± 11.5%	74.9% ± 14.2%	83.4% ± 3.6%
T2 Only	63.7% ± 11.7%	53.3% ± 13.8%	76.2% ± 4.3%
T3 Only	57.6% ± 12.2%	56.0% ± 9.8%	76.3% ± 6.3%
T4 Only	82.3% ± 10.1%	76.5% ± 12.9%	83.3% ± 4.4%
T5 Only[♭]	84.0% ± 9.7%	79.2% ± 13.1%	83.5% ± 3.2%

[♭] T5-only applies exponential smoothing to T1–T4 without a learned model. It is one hand-coded temporal baseline, not an upper bound on temporal smoothing.

block in Table 4. The two findings are complementary. Cross-channel combination explains the gap to single-branch baselines; the shape block explains the gap to the marginal learned baselines.

4.9 Scalability and Sensitivity Analysis

We also vary the number of generated nodes from 5 to 50. Accuracy is empirical; time and communication values are formula outputs rather than measurements.

Table 18: Generated-node-count sensitivity under a 30% malicious-node ratio. Time and communication columns are formula outputs, not measurements.

UAV count N	Seed parameter M	Accuracy (%)	Time model (ms)	Cost model (KB/UAV)
5	2	97.2 ± 7.0	36.5 ± 3.0	27.1 ± 1.9
15	5	97.1 ± 4.3	58.3 ± 4.3	34.0 ± 1.8
30	10	93.5 ± 3.8	95.1 ± 4.2	44.9 ± 1.9
50	17	95.2 ± 2.9	152.0 ± 3.8	59.8 ± 1.7

Detection accuracy stays above 93% on the generated size grid. The apparent growth in time and communication is predetermined by `resource_metrics`; no scaling law can be inferred from those columns.

Sensitivity analysis is limited to the fitted architecture, fixed threshold, and one trust-stream generator. Loss weights, temporal windows, threshold calibration, and distribution shift have not been studied systematically.

4.10 Trust Factor Correlation Analysis

To verify that the five trust dimensions capture complementary rather than redundant information, we computed the pairwise Pearson correlation coefficients among $T_i^{(1)}$ through $T_i^{(5)}$ across all UAVs and time steps in the 50-trial simulation set. The correlation matrix (averaged over trials) is:

The strongest correlations involve the historical trust dimension T5, especially with T4 (0.65 ± 0.09) and T1 (0.64 ± 0.08), which is expected because T5 is an exponential history of recent trust evidence.

Table 19: Pairwise Pearson correlations among the five trust dimensions (mean ± std over 50 trials).

	T1 (Behavior)	T2 (Comm.)	T3 (Energy)	T4 (Task)	T5 (History)
T1	1.00	0.28 ± 0.10	0.11 ± 0.08	0.41 ± 0.10	0.64 ± 0.08
T2	–	1.00	0.13 ± 0.09	0.27 ± 0.10	0.51 ± 0.07
T3	–	–	1.00	0.11 ± 0.09	0.34 ± 0.09
T4	–	–	–	1.00	0.65 ± 0.09
T5	–	–	–	–	1.00

Among the instantaneous dimensions T1–T4, all correlations remain at or below 0.41. Thus, T5 should be interpreted as a temporal memory channel rather than an independent sensor, while the instantaneous factors retain enough diversity to justify multi-factor fusion.

5 Conclusions

This study provides a reproducible trust-stream benchmark and a spatio-temporally augmented risk model for UAV task allocation. Its principal result is that the summarisation step that conventional trust aggregation performs has two structural blind spots, in two independent directions, and that a five-dimensional block closes each of them. The first is temporal: across five rule-based comparators on–off detection is 0.0–38%, and the same models reach 88.0–92% once five scale-free temporal-shape statistics are added, a +50.0 to +92.0 pp gap that appears only on the attack class the aggregators are structurally unable to express. The second is spatial, and no amount of temporal detail addresses it: an attacker whose behavior is clean is invisible to every aggregator that scores one node at a time, and with the marginal and shape blocks alone it is detected at 2.1%, against 6.2% and 4.2% for the learned per-node baselines. Five neighborhood-relative statistics raise the same model to 83.6% on the same seeds, a +81.5 pp paired gain ($t = 63.7$, $p < 0.0001$), and the two blocks are disjoint modalities rather than redundant descriptions of one. On held-out seeds from the published generator S-TIM reaches 97.1% accuracy and a 95.7% F-score, exceeding the strongest rule-based comparator by +9.0 pp and +15.0 pp on paired seeds ($p < 10^{-6}$), and in the spatially inconsistent scenario, where the published generator's perfect separability no longer compresses the comparison, it reaches 93.8% accuracy, 89.4% F-score and 88.9% TSR, against 86.0%, 72.8% and 82.2% for the strongest learned control and 39.7–44.0%, 44.9–45.5% and 74.5–75.9% for the rule adapters, with paired differences of +7.7 pp, +16.7 pp and +6.7 pp against the learned control ($p < 0.0001$). At the aggregate level every learned method sits above 97%, so the aggregate metric compresses the method classes and the separation is reported where it resolves, in Table 5 by attack type. There S-TIM matches a same-feature single-head MLP to within 0.4 pp ($p = 0.51$), which locates the binding constraint in the input rather than in the network and is what a representation-level result requires. Three consequences follow. First, the gain is located in the representation rather than in the encoder, and the controls in Section 4.7 establish it: enlarging the training set sixfold, sweeping the decision threshold across its full range, and replacing the auxiliary targets with attack-modality supervision that the binary label cannot express all leave the detection metrics unchanged, and an information probe shows why, because the auxiliary target is already 96.7% recoverable from the marginal features and a head trained on it can only re-encode information the risk head already receives. Second, and in the opposite direction from what the architecture implies, the *representation* is not neutral. Conventional trust aggregators summarise each channel over a fixed window, which is exactly the statistic a periodic attacker is built to survive: with only those summaries, on–off detection remains at 0.0–38% for the rule adapters. Adding five scale-free shape statistics lifts on–off detection in every learned method tested, in a logistic regression and in a network alike, and a time-shuffle control attributes +10.6 pp of AUC to sequence order rather than to added dimensionality. The same holds in the spatial direction, and more sharply: a node that reports well-formed telemetry is summarised exactly like a benign one, so the missing information has to come from a comparison against the neighbourhood rather than from the node. Third, the blocks are not free, and the prices are reported with the gains. The shape block costs 8.7 pp on constant attackers, and the full 23-dimensional representation still costs 3.6 pp on constant attackers and 2.2 pp on dormant ones against the 13-dimensional one, together with 1.3 pp of benign specificity. The classes that lose a few points are the ones window aggregation already

handled; the classes that gain by tens of points are the ones it structurally cannot express. The principal empirical result is therefore not that this generator is separable by a learned nonlinear classifier—that was already true—but that the standard summarisation used by the trust-aggregation literature is the specific thing that fails on intermittent attackers and on behaviorally-clean attackers, and that once the representation is repaired the choice of classifier stops mattering.

The packet-level study is reported with the same discipline. On five multi-seed ns-3 traces of forty UAVs four of the five rule aggregators are not inferior to the proposed model, 0.99 accuracy for MFA and GDM-DTM against 0.93 for S-TIM, because the attacker in that scenario is a level shift and a threshold on the level is what a window aggregator computes. The scenario contains no level-indistinguishable attacker, so it neither tests nor refutes the two blind-spot claims, and the honest summary of the packet evidence is that the claims are untested there rather than confirmed. Two further facts are stated without embellishment: the learned models' decision threshold does not transfer to the packet domain, which costs S-TIM recall while its ranking quality stays high, at an AUC of 0.966 against 1.000 for the feature-matched control, and the attack parameters of the scenario were not held out from training. Against the feature-matched control on identical features, at the published operating point, the proposed model leads by +4.5 pp accuracy ($p = 0.037$) and +10.9 pp F-score ($p = 0.055$).

The 89.3% task-success value is a simulator-specific proxy. Since the allocation code applies S-TIM-specific multipliers and does not use the learned incentive head, the 0.3 pp deficit against MLP-Trust falls well inside the seed-level standard deviation and cannot support a causal claim about incentive learning. Likewise, the code-level ablations do not isolate named architectural components. The packet-level evidence is now five multi-seed traces rather than one short trace, and its limits are of a different kind. On these traces four of the five rule aggregators are not inferior: MFA and GDM-DTM reach 0.99 accuracy and 0.98 F-score against 0.93 and 0.86 for S-TIM, because the attacker in that scenario is a level shift—a larger path deviation, a faster energy drain, fewer completed tasks—and a threshold on the level is exactly what a window aggregator computes. The attack class this manuscript is about is the opposite one, whose level is indistinguishable from the benign population and whose temporal shape is the only evidence, and that class is not present in the packet scenario, so the packet study neither tests nor refutes the two blind-spot claims. Three further limits apply. First, the learned models' decision threshold does not transfer: S-TIM's recall on the traces is 0.767 at a precision of 1.000 while its AUC is 0.966 and MLP-Trust's is 1.000, so the information is present in the packet domain and the fixed cut is what fails, and a learned model that has to be re-thresholded in a new domain is a real cost. Second, the attack parameters of the scenario were not held out from training, so the study is a domain-transfer measurement rather than a generalization test. Third, one input column, the coefficient of variation of the switch intervals, is constant across nodes in every trace, because the scenario contains only periodically triggered attackers; it is disclosed rather than used to drop a trace. The single 49-step trace that the artifact originally shipped remains in Table 10 as a regime diagnosis: at that length the converter leaves a 40-step post-warm-up window, the shape block is defined only for windows of at least 122 steps, and six of the eighteen input columns degenerate to constants at standardised values far outside the training range, so its numbers measure an input-format mismatch rather than detector robustness in either direction.

A further caveat concerns the generator itself. Because the benign reference profile is separated from each attack profile by roughly four standard deviations before camouflage is applied, a large part of the achievable accuracy follows from the synthetic profile design rather than from any property a deployed detector would need. The camouflage mechanism in the generator is what prevents this separation from being trivial; it is reported here as a limitation of the current evaluation rather than as a validated adversarial model.

That separability removes the model itself as a variable, which explains the ceiling in Table 3. Enlarging the training set sixfold, from 450 to 2700 node-level samples over sixty seeds, leaves the training-split separation exact: accuracy 1.000, false-positive rate 0.000, mean risk 0.9999 on malicious nodes and 0.0002 on benign ones. Once a classifier separates the training split perfectly, additional capacity and additional tasks cannot buy it anything, and every learned method must converge on the same decision boundary and differ only in how that boundary happens to fall on held-out seeds. That is the mechanism behind the S-TIM/MLP-Trust equality, a property of the benchmark rather than a defect of either model, and it locates where the paper's gain lives: the entire spread between the rule adapters at 87.6% and the learned methods at 96.8% is attributable to the *input representation*, and within the learned class the classifier is not the binding constraint. The spatial scenario of Section 4.6 is built so that this ceiling does not apply, because the attack class it contains is separable only by a block that the per-node learners do not receive. There the separation reappears, and it is again located in the representation: given the

identical 23-dimensional vector, the single-head MLP and the multi-head policy are equivalent, while the linear head extracts less of the neighborhood signal and pays 6.3 pp on the classes it already handled. A benchmark in which architecture matters would need harder separation, and Table 20 measures what that would take, because the obvious knob does not work.

How hard would the benchmark have to be? The question a reviewer asks next is how much harder the benchmark would have to be for the comparison in Table 3 to mean something. The generator's camouflage factor is the obvious knob, and it is worth stating what it does, because the answer is not the intuitive one. The factor mixes a malicious node's attack profile with an evasive reference, so a *larger* factor is a *quieter* attacker, and the effect size that matters is the one on the window summary rather than on a single step, because a window averages the observation noise away. Moving the factor from its published value to its quietest level takes that effect size from 1.55 to 0.36, which is below the threshold at which an effect is conventionally called negligible, and the marginal block's held-out separability still does not fall below 93%.

The reason is that the marginal block is not only a level summary. It also carries the recent mean, the slope, the volatility and the history channel, and a periodic attacker's intermittency is visible in all of them, so removing the level does not remove the attacker. The second knob is the benign population's own spread, the per-node offset the generator applies to every node, which the camouflage factor does not touch at all; widening it until the attacker's level falls inside the benign distribution takes the effect size to 0.49 and the separability to 91.7%, and combining the two reaches 0.36 and 92.8%. Neither is the regime in which a per-node summary fails, and the eighteen-dimensional separability is no lower than the thirteen-dimensional one at every point, so the shape block adds no discriminative power to a label that is already separable.

A benchmark of this family therefore does not run out of separability by being made quieter. It runs out only when the attacker's evidence is not in its own channels at all, which is the case the spatial scenario of Section 4.6 constructs, and the only place in this manuscript where the method classes separate. This is a limitation of the generator as much as of the method, and it is reported as such: the measurements below are a diagnosis of where the comparison can and cannot be made, not a claim that a harder benchmark would favour the proposed model.

Table 20: How far the benchmark can be made harder before the comparison means something. The level effect size is Cohen's d between the active attackers' windowed level and the benign windowed level, averaged over the four behaviour channels, on the window the marginal block uses, so it is the effect size a window summary can actually exploit. The separability is the held-out accuracy of a probe on the marginal features, trained on forty-eight seeds and evaluated on twelve, and the last column repeats it with the shape block added.

Configuration	mean factor	level effect size	marginal	with shape
published, camouflage $\beta(2.7, 1.8)$	0.600	1.55	97.2%	97.8%
camouflage $\beta(6, 1)$	0.857	0.90	93.9%	95.0%
camouflage $\beta(9, 1)$	0.900	0.81	96.1%	95.6%
node offset 0.40	0.600	0.56	93.3%	96.1%
node offset 0.60	0.600	0.49	91.7%	93.9%
node offset 0.40 and camouflage $\beta(9, 1)$	0.900	0.36	92.8%	95.0%

Reproducible from `artifacts/analysis/camouflage_direction_sweep.py`. The node offset is the per-node, per-channel constant the generator applies to every node, and it is the one part of the level evidence the camouflage factor leaves untouched; it is now a configuration key, `generator.node_bias_sd`, whose default reproduces the published generator exactly.

The repository is nevertheless useful as an auditable starting point. A defensible next study should replace the synthetic allocation rules with a common policy for all detectors, measure actual runtime and messages, implement genuine agent-local observations and aggregation, and repeat the multi-seed packet-trace study with attack parameters held out from training and with a scenario whose attacker is level-indistinguishable, so that the two blind-spot claims are tested on packet data rather than only on generated streams. Reinforcement-learning claims would additionally require environment interaction, reward-defined returns, and comparison with supervised and non-learning policies. Until those experiments are available, S-TIM should be described as a supervised trust-stream proof of concept rather than a validated MARL or zero-trust deployment.

Author Contributions

All authors contributed to the conceptualization, methodology, analysis, and manuscript preparation. Author roles will be specified in the camera-ready version.

Funding

This research received no external funding.

Institutional Review Board Statement

Not applicable.

Informed Consent Statement

Not applicable.

Data Availability Statement

The replication package is included under `artifacts/`. It provides the code, configurations, trained models, evaluation outputs, statistical summaries, generated figures, baseline reimplementations, and packet-trace conversion materials needed to reproduce the trust-layer results reported in this manuscript. The packet-trace material contains the single short trace and the five multi-seed traces, together with the per-trace metrics and the per-trace regime verdicts; the converted trust streams are regenerated from the raw traces by the committed converter, and the simulator build that produced the five traces is recorded in `artifacts/ns3/ENVIRONMENT.md`.

Conflicts of Interest

The authors declare no conflict of interest.

References

[1] Kundu, J.; Alam, S.; Das, J.C.; Dey, A.; Bari, S. Trust-Based Flying Ad Hoc Network: A Survey. *IEEE Access* **2024**, *12*, 99258–99281.

[2] Noor, F.; Khan, M.A.; Al-Zahrani, A.; Ullah, I.; Al-Dhlan, K.A. A Review on Communications Perspective of Flying Ad-Hoc Networks: Key Enabling Wireless Technologies, Applications, Challenges and Open Research Topics. *Drones* **2020**, *4*, 65.

[3] Brik, B.; Ksentini, A.; Bouaziz, M. Federated Learning for UAVs-Enabled Wireless Networks: Use Cases, Challenges, and Open Problems. *IEEE Netw.* **2020**, *34*, 144–150.

[4] Nguyen, D.C.; Ding, M.; Pathirana, P.N.; Seneviratne, A.; Li, J.; Poor, H.V. Federated Learning for Internet of Things: A Comprehensive Survey. *IEEE Commun. Surv. Tutor.* **2021**, *23*, 1622–1658.

[5] Qu, Y.; Dai, H.; Zhuang, Y.; Chen, J.; Dong, C.; Wu, F.; Guo, S. Decentralized Federated Learning for UAV Networks: Architecture, Challenges, and Opportunities. *IEEE Netw.* **2021**, *35*, 156–162.

[6] Zuo, J.; Cao, R.; Qi, J.; Gao, P.; Wang, Z.; Li, J.; Zhang, L.; Lu, Y. A Hierarchical Blockchain-Based Trust Measurement Method for Drone Cluster Nodes. *Drones* **2023**, *7*, 627.

[7] Ge, C.; Zhou, L.; Hancke, G.P.; Su, C. A Provenance-Aware Distributed Trust Model for Resilient Unmanned Aerial Vehicle Networks. *IEEE Internet Things J.* **2021**, *8*, 12481–12489.

[8] Benfriha, S.; Labraoui, N.; Bensaid, R.; Bany Salameh, H.; Saidi, H. FUBA: A Fuzzy-Based Unmanned Aerial Vehicle Behaviour Analytics for Trust Management in Flying Ad-Hoc Networks. *IET Networks* **2024**, *13*, 208–220.

[9] Li, H.; Li, X.; Dunkin, F.; Zhang, Z.; Lu, X. Adaptive Multi-Granularity Trust Management Scheme for UAV Visual Sensor Security under Adversarial Attacks. *Comput. Secur.* **2025**, *148*, 104108.

[10] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. GALTrust: Generative Adversarial Learning-Based Framework for Trust Management in Spatial Crowdsourcing Drone Services. *IEEE Trans. Consum. Electron.* **2024**, *70*, 6196–6207.

[11] Ceviz, O.; Sadioglu, P.; Sen, S.; Vassilakis, V.G. A Novel Federated Learning-Based IDS for Enhancing UAVs Privacy and Security. *Internet Things* **2025**, *31*, 101592.

[12] Lu, Y.; Yang, T.; Zhao, C.; Chen, W.; Liu, Y.; Zeng, R. A Swarm Anomaly Detection Model for IoT UAVs Based on a Multi-Modal Denoising Autoencoder and Federated Learning. *Comput. Ind. Eng.* **2024**, *196*, 110454.

[13] Gad, G.; Farrag, A.; Aboulfotouh, A.; Bedda, K.; Fadlullah, Z.M.; Fouda, M.M. Joint Self-Organizing Maps and Knowledge-Distillation-Based Communication-Efficient Federated Learning for Resource-Constrained UAV-IoT Systems. *IEEE Internet Things J.* **2024**, *11*, 34270–34282.

[14] Wang, X.; Yuan, Q.; Liu, C.; Wang, P.; Tao, J.; Li, L.; Wang, X.; Wang, Y. Who Is the Wolf in Sheep's Clothing? A Context-Aware Trust Evaluation Model for Malicious UAV Detection during Authentication. *Comput. Netw.* **2026**, *279*, 112145.

[15] Rose, S.; Borchert, O.; Mitchell, S.; Connelly, S. Zero Trust Architecture. *NIST Spec. Publ.* **2020**, *800-207*, 1–59.

[16] Ekechi, C.C.; Elfouly, T.; Alouani, A.; Khattab, T. A Survey on UAV Control with Multi-Agent Reinforcement Learning. *Drones* **2025**, *9*, 484.

[17] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. Digital Twin-Driven Trust Management in Open RAN-Based Spatial Crowdsourcing Drone Services. *IEEE Trans. Green Commun. Netw.* **2024**, *8*, 1061–1075.

[18] Abubaker, Z.; Javaid, N.; Almogren, A.; Akbar, M.; Zuair, M.; Ben-Othman, J. Blockchained Service Provisioning and Malicious Node Detection via Federated Learning in Scalable Internet of Sensor Things Networks. *IEEE Trans. Netw. Serv. Manag.* **2023**, *20*, 1847–1865.

[19] Akram, J.; Umair, M.; Jhaveri, R.H.; Riaz, M.; Chi, H.R.; Malebary, S. Chained-Drones: Blockchain-Based Privacy-Preserving Framework for Secure and Intelligent Service Provisioning in Internet of Drone Things. *Comput. Electr. Eng.* **2023**, *110*, 108772.

[20] Zheng, J.; Xu, J.; Du, H.; Niyato, D.; Kang, J.; Nie, J.; Wang, Z. Trust Management of Tiny Federated Learning in Internet of Unmanned Aerial Vehicles. *IEEE Internet Things J.* **2024**, *11*, 21046–21060.

[21] Tong, Z.; Wang, J.; Hou, X.; Chen, J.; Jiao, Z.; Liu, J. Blockchain-Based Trustworthy and Efficient Hierarchical Federated Learning for UAV-Enabled IoT Networks. *IEEE Internet Things J.* **2024**, *11*, 15504–15522.

[22] Xu, Q.; Li, R.; Qi, Y.; Su, Z.; Fang, D. Trust-Enhanced Game Incentive for Secure Quantum Federated Learning in UAV-Assisted Wireless Networks. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2841–2856, doi: 10.1109/JSAC.2025.3568054.

[23] Hu, Y.; Gan, Y.; Wu, H.; Wang, C.; Ma, M.; Xiong, C. GDM-DTM: A Group Decision-Making-Enabled Dynamic Trust Management Method for Malicious Node Detection in Low-Altitude UAV Networks. *Sensors* **2025**, *25*, 3982.

[24] Kumar, P.; Rayudu Tummala, V.M.; Roy, A. TOFU: Trust-Based Task Offloading in UAV-as-a-Service. *IEEE Trans. Veh. Technol.*, 2025, doi: 10.1109/TVT.2025.3604589.

[25] Kannan, S.; Venkataraman, R.; Ramachandran, G.S. On-Off Attack Detection in Trust Model Using Intra-Daily Variability for the IoT. *Bull. Electr. Eng. Inform.* **2023**, *12*, doi: 10.11591/eei.v12i6.6110.

[26] Chae, Y.; DiPippo, L.C.; Sun, Y.L. Trust Management for Defending On-Off Attacks. *IEEE Trans. Parallel Distrib. Syst.* **2014**, *26*, 1178–1191, doi: 10.1109/TPDS.2014.2317719.

[27] Mendoza, C.V.L.; Kleinschmidt, J.H. Mitigating On-Off Attacks in the Internet of Things Using a Distributed Trust Management Scheme. *Int. J. Distrib. Sens. Netw.* **2015**, *11*, 859731, doi: 10.1155/2015/859731.

[28] Caminha, J.; Perkusich, A.; Perkusich, M. A Smart Trust Management Method to Detect On-Off Attacks in the Internet of Things. *Secur. Commun. Netw.* **2018**, *2018*, 6063456, doi: 10.1155/2018/6063456.

[29] Salimitari, M.; Bhattacharjee, S.; Chatterjee, M.; Fallah, Y.P. A Prospect Theoretic Approach for Trust Management in IoT Networks Under Manipulation Attacks. *ACM Trans. Sens. Netw.* **2020**, *16*, 26, doi: 10.1145/3392058.

[30] Ziauddin, M.I.; Rabbi, R.R.; Mehrab, S.M.; Faiyaz, F.; Jahan, M. An Efficient Recommendation Filtering-Based Trust Model for Securing Internet of Things. *SN Comput. Sci.* **2026**, *7*, 581, doi: 10.1007/s42979-026-05087-1.

[31] Wang, X.; Yuan, Q.; Wang, Y.; Teng, J.; Tao, J.; Shen, M. Zero Trust-Driven Collaborative Intrusion Detection in Internet of Things: A Continuous Trust Assessment Approach. *IEEE Internet Comput.* **2025**, *29*, 37–47.

[32] Al Ridhawi, I.; Aloqaily, M. Zero-Trust UAV-Enabled and DT-Supported 6G Networks. *IEEE Internet Comput.* **2023**, *27*, 59–67.

[33] Wang, L.; Wang, K.; Pan, C.; Xu, W.; Aslam, N.; Hanzo, L. Multi-Agent Deep Reinforcement Learning Based Trajectory Planning for Multi-UAV Assisted Mobile Edge Computing. *IEEE Trans. Cogn. Commun. Netw.* **2021**, *7*, 73–84.

[34] Nie, Y.; Zhao, J.; Gao, F.; Yu, F.R. Semi-Distributed Resource Management in UAV-Aided MEC Systems: A Multi-Agent Federated Reinforcement Learning Approach. *IEEE Trans. Veh. Technol.* **2021**, *70*, 13162–13173.

[35] Alam, M.M.; Moh, S. Joint Optimization of Trajectory Control, Task Offloading, and Resource Allocation in Air-Ground Integrated Networks. *IEEE Internet Things J.* **2024**, *11*, 24273–24288.

[36] Yang, H.; Zhao, J.; Xiong, Z.; Lam, K.-Y.; Sun, S.; Xiao, L. Privacy-Preserving Federated Learning for UAV-Enabled Networks: Learning-Based Joint Scheduling and Resource Management. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 3144–3159.

[37] Lim, W.Y.B.; Huang, J.; Xiong, Z.; Kang, J.; Niyato, D.; Hua, X.-S.; Leung, C.; Miao, C. Towards Federated Learning in UAV-Enabled Internet of Vehicles: A Multi-Dimensional Contract-Matching Approach. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 5140–5154.

[38] Wang, S.; Hosseinalipour, S.; Gorlatova, M.; Brinton, C.G.; Chiang, M. UAV-Assisted Online Machine Learning Over Multi-Tiered Networks: A Hierarchical Nested Personalized Federated Learning Approach. *IEEE Internet Things J.* **2024**, *11*, 20899–20911.

[39] Qureshi, K.I.; Wang, L.; Xiong, X.; Lodhi, M.A. Asynchronous Federated Learning for Resource Allocation in Software-Defined Internet of UAVs. *IEEE Internet Things J.* **2024**, *11*, 15523–15538.

[40] Jing, Y.; Qu, Y.; Dong, C.; Ren, W.; Shen, Y.; Wu, Q.; Guo, S. Exploiting UAV for Air-Ground Integrated Federated Learning: A Joint UAV Location and Resource Optimization Approach. *IEEE Trans. Green Commun. Netw.* **2023**, *7*, 1420–1433.

[41] Fu, Z.; Liu, J.; Mao, Y.; Qu, L.; Xie, L.F.; Wang, X. Energy-Efficient UAV-Assisted Federated Learning: Trajectory Optimization, Device Scheduling, and Resource Management. *IEEE Trans. Netw. Serv. Manag.* **2025**, *22*, 974–988.

[42] Cao, X.; Nan, G.; Guo, H.; Mu, H.; Wang, L.; Lin, Y.; Zhou, Q.; Li, J.; Qin, B.; Cui, Q.; Tao, X.; Fang, H.; Du, H.; Quek, T.Q.S. Exploring LLM-Based Multi-Agent Situation Awareness for Zero-Trust Space-Air-Ground Integrated Network. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2230–2247.

[43] Bertsekas, D.P.; Tsitsiklis, J.N. *Neuro-Dynamic Programming*; Athena Scientific: Nashua, NH, USA, 1996.

[44] Oliehoek, F.A.; Spaan, M.T.; Vlassis, N. Optimal and Approximate Solutions to Decentralized POMDPs. In *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS)*; AAAI Press: 2008; pp. 274–281.

[45] Riley, G.F.; Henderson, T.R. The ns-3 Network Simulator. In *Modeling and Tools for Network Simulation*; Wehrle, K.; Günes, M.; Gross, J., Eds.; Springer: Berlin, Germany, 2010; pp. 15–34.

[46] Hausknecht, M.; Stone, P. Deep Recurrent Q-Learning for Partially Observable MDPs. In *Proceedings of the AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*; AAAI Press: 2015; pp. 29–37.

[47] Hausknecht, M.; Stone, P. Deep Reinforcement Learning in Parameterized Action Space. In *Proceedings of the International Conference on Learning Representations (ICLR)*; 2016.

[48] Xiong, J.; Wang, Q.; Yang, Z.; Sun, P.; Han, L.; Zheng, Y.; Fu, H.; Zhang, T.; Liu, J.; Liu, H. Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space. *arXiv* **2018**, arXiv:1810.06394.

[49] Vijay, V.; Pant, K.A.; Cho, M.; Guo, Y.; Goppert, J.M.; Hwang, I. Range-Based Multi-Robot Integrity Monitoring For Cyberattacks and Faults: An Anchor-Free Approach. *IEEE Robot. Autom. Lett.* **2025**, *10*, 2630–2637, doi: 10.1109/lra.2025.3534068.

[50] Zhong, L.; Tang, T.; Li, R.; Xia, Z.; Tang, M. Enhancing the Resilience of UAV Networks Against GPS Spoofing Attacks via Byzantine Distributed Detection Algorithm. *Reliab. Eng. Syst. Saf.* **2026**, *269*, 112101, doi: 10.1016/j.ress.2025.112101.

[51] Dang, Y.; Benzaid, C.; Yang, B.; Taleb, T.; Shen, Y. Deep-Ensemble-Learning-Based GPS Spoofing Detection for Cellular-Connected UAVs. *IEEE Internet Things J.* **2022**, *9*, 25068–25085, doi: 10.1109/jiot.2022.3195320.

[52] Fang, Z.; Han, B.; Schotten, H.D. Trustworthy UAV Cooperative Localization: Information Analysis of Performance and Security. *IEEE Trans. Veh. Technol.* **2025**, *74*, 12997–13012, doi: 10.1109/tvt.2025.3555490.

[53] Zhao, S.; Fu, J.; Li, B. A Review of GNSS Spoofing Detection Based on Multi-Source Fusion. *Electronics* **2026**, *15*, 3326, doi: 10.3390/electronics15153326.

[54] Tariq, U.; Ahamed Ahanger, T. Multi-Model Fusion for Robust Detection and Resilient Mitigation of BeiDou Spoofing in Decentralized UAV Swarm Systems. *PeerJ Comput. Sci.* **2026**, *12*, e3875, doi: 10.7717/peerj-cs.3875.

[55] Keshavarz, M.; Shamsoshoara, A.; Afghah, F.; Ashdown, J. A Real-Time Framework for Trust Monitoring in a Network of Unmanned Aerial Vehicles. In *Proceedings of the IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*; IEEE: 2020; pp. 677–682, doi: 10.1109/infocomwkshps50562.2020.9162761.